Private Federated Aggregation of LLM-Agent Memory: Usable Shared Memory with a Measurable Leakage-Drop Guarantee under Secure Aggregation and the Skellam Mechanism

Nikhil Sharma[1*]

Corresponding author(s). E-mail(s): nikmsharma@gmail.com;

Abstract

LLM agents accumulate per-user memory — distilled notes and preferences — that makes them personal and effective. Pooling it across users would give a fleet a shared **routing prior** over what the fleet collectively knows, but memory is exactly the sensitive artifact: published extraction and membership-inference attacks recover verbatim user facts from it.

We aggregate per-user memory-note embeddings into a bucketed **centroid memory** under **Secure Aggregation** composed with the discrete **Skellam mechanism**, so the pool carries a central differential-privacy guarantee with no trusted curator and no note in the clear. We evaluate on real long-horizon conversational memory and on LLM-distilled agent notes, insisting that utility and leakage be read at the same operating point.

Four findings. First, on a **joint frontier** measuring both axes at every bucket fidelity, the coarse regime is usable and safe at once: the pool keeps **95% of clean retrieval utility** while a calibrated likelihood-ratio attack falls from an AUC of 0.833 to the 1% false-positive floor. Second, that endpoint is **invisible to the weak attack the literature defaults to**: an uncalibrated cosine proxy reports chance on the very release the calibrated attack breaks, so a proxy-only evaluation would certify a leaking pool as safe. Third, **data-dependent preprocessing silently inflates such results**: fitting the projection on user notes is an unaccounted release that also manufactures an apparent dimension effect, which vanishes under a public-seed projection. Fourth, we account for the **repeated release** a deployed memory requires, whose cost grows only as the square root of the round count. Every parameter of the mechanism is public or DP-accounted, so the reported budget covers the whole pipeline.

Keywords: Differential privacy, Secure aggregation, Skellam mechanism, LLM agent memory, Federated learning, Membership inference

1 Introduction

Agent memory is the new sensitive payload.

Production LLM-agent frameworks such as A-MEM [1] and Mem0 [2] persist a per-user store of distilled notes: "the user is allergic to penicillin," "prefers metric units," "is migrating a Postgres 14 cluster." This memory is what makes an agent personal, and it is also a dense concentrate of personally identifying information. A natural, high-value idea is to **share memory across users**, so that a fleet of agents can profit from what the fleet collectively knows. But naïvely pooling raw notes is a privacy disaster, and recent work shows the threat is concrete rather than hypothetical: **MEXTRA** [3] extracts verbatim memory content, and **MRMMIA** [4] performs membership inference against agent memory with high success.

What a private pool can and cannot be: a routing prior, not a note exchange.

We are precise about the object we release, because it bounds what may honestly be claimed for it. The shared pool is K *centroid vectors* and **no text**. A receiving agent therefore cannot read another user's note out of it, and we do not claim it can: verbatim knowledge transfer across users is *not* what this mechanism delivers, and any mechanism that did deliver it would be releasing the very artifact the attacks of §2 consume. What the pool *does* deliver is a **shared routing prior**: a fleet-level map of *which regions of memory space hold knowledge*, against which any agent can route a query — to the right local memory, the right tool, the right public corpus — without any note leaving its device. That is a narrower promise than "every agent inherits every fix," and it is the one our metric actually measures (evidence-recall: did the query reach the bucket containing its evidence?). It is also, we argue, the promise worth keeping: the coarse pool is exactly the part of shared memory that *can* be released under a guarantee, and §7.1 shows it is released nearly for free.

Why the obvious defenses are insufficient.

Access-control layers (*Collaborative Memory* [5]) and on-device masking (*MemPrivacy* [6]) restrict *who* sees a note but provide no formal guarantee over the *aggregate*, and a curated central datastore [7] reintroduces a trusted party. What is missing is a mechanism that (a) needs **no trusted curator**, (b) gives a **central-DP guarantee over the shared pool**, and (c) is shown to actually **degrade the known attacks** while keeping the pool useful.

Our approach.

We treat federated memory aggregation as a **secure summation of per-user bucketed embeddings**. Each user projects their notes to a working dimension d with a **public-seed random projection**, maps them into K shared buckets (random-hyperplane LSH), forms a per-bucket sum-vector, and contributes it through **SecAgg** [8] so the server sees only the masked sum; the **Skellam mechanism** [9] adds calibrated discrete noise that **composes under summation**, yielding a single distributed-DP guarantee on the pooled centroid memory. Retrieval is cosine lookup

against the noised centroids. The cryptographic path is used **verbatim** from federated discrete-DP work; the novelty is the *payload* (agent-memory embeddings) and the *evaluation* (a measured attack-success drop).

Why this payload, and why now.

Federated DP for LLM *adaptation* (DP-LoRA, prompt tuning) is saturated, and recent federated-privacy work for LLMs targets *other* objects — DP synthetic-data generation (POPri [10]) and privacy-constrained agent self-evolution (Fed-SE [11]) — rather than the aggregation of a shared **memory** pool; contemporaneous agent-memory security surveys [12] catalogue attacks and access-control/redaction defenses but no curator-free DP aggregation mechanism. The nearest mechanism, *DP Datastore Generation* [7], privatises a retrieval datastore but in a **centralized, single-curator** setting with generic additive noise and no secure aggregation (§2). Federated agent-*memory* aggregation under SecAgg + discrete Skellam DP is thus an open seam. Crucially, agent-memory objects are **genuinely tiny-dimensional and often already discrete**, which is exactly the regime where the discrete Skellam mechanism is most efficient and where our dimension analysis is most credible.

Contributions

1. **A federated agent-memory aggregation mechanism** that composes SecAgg with the Skellam mechanism over bucketed memory-note embeddings, giving a curator-free central-DP shared routing prior (§4–§5).
2. **A joint utility/leakage frontier, and a positive endpoint on it.** We measure both axes **at every fidelity** K, which a two-axis claim requires and which our own superseded draft did not do (it reported utility only at $K \leq 256$ and leakage only at $K \geq 512$, so its two headline numbers described disjoint releases). On the frontier the coarse-K regime is usable *and* safe at one operating point: at $K = 32$ the pool retains **95% of clean evidence-recall** at $\varepsilon \approx 9.3$ while a calibrated offline-LiRA attack falls from **AUC 0.833 / TPR@1%FPR 0.175** to the **1% false-positive floor** (0.011), and a reconstruction-decode attack from 0.771 to 0.396; at $K = 128$ (72% retention) *both* attacks reach their floor (§7.1).
3. **A methodological result: the weak attack the literature defaults to is blind exactly where it matters.** The uncalibrated cosine proxy used by prior agent-memory work reads **0.504 — chance — at** $K = 32$, on the same clean release the calibrated attack breaks at AUC 0.833. A proxy-only evaluation would therefore have certified a leaking pool as safe, and would have reported (as our superseded draft did) that leakage only exists in a sparse regime nobody would deploy. The endpoint is real; the proxy could not see it (§7.5).
4. **A negative result with teeth: the dimension "crossover" is an artifact of data-dependent preprocessing.** Fitting the projection on user notes is an unaccounted release *and* it fabricates the finding that tiny-d is jointly optimal for privacy and utility. Under a data-independent projection the effect **vanishes on both axes** — no ordering among d survives the seed noise on the utility axis, and the pre-DP leakage gradient flattens — with an identity-projection control

($d = 384$) that all three projections must and do agree on (§7.4). We believe this confound is not specific to this paper.

5. **An end-to-end-accounted mechanism, including repeated release**: the projection, the LSH anchors, the clip bound C and the quantiser bound $B := C$ are all public or DP-selected, so the reported ε covers the entire pipeline rather than only its last stage (§5.2, §7.7); the clip-selection cost is 0.2% of the budget. Because agent memory is *re-aggregated* as users write notes, we also account for the T-round release that a deployment actually performs: naive re-release is ruinous (ε 9.3 $\to$ 93 for a month of daily rounds), but the cost is $\sqrt{T}$ in the noise scale, so a **monthly cadence sustained for a year fits inside the same $\varepsilon \approx 9.3$ at 81% retention** (§5.7).
6. **A rigorous, tightened accounting**: an exact Skellam-RDP ε for the discrete mechanism (Agarwal et al. Thm. 3.5 [9]), a proof that **discretisation is free** at our resolution, and a **vector-only release** that gives identical retrieval at $\sim$1.5$\times$ tighter ε because the count magnitude cancels under normalisation. The count channel's one non-redundant use is *occupancy*, and we show it **cannot pay for itself**: no occupancy rule is simultaneously free and accurate, because occupancy and membership are the same signal (§5.3, §7.11).
7. **A realistic-payload validation, on the calibrated attack**: we run the LiRA *directly on* LLM-distilled A-MEM/Mem0-style notes — the object a production agent actually stores — and the endpoint holds at the same operating point and the same budget as on raw turns (clean AUC 0.838, TPR@1%FPR 0.171; both at the floor under DP). The uncalibrated proxy had suggested distilled notes leak *more* than raw turns; calibration shows they do not, and we retract that reading rather than keep the more dramatic one (§7.6).

Utility and proxy-leakage results are **5-seed means with final (not peak) metrics**, reported with their standard deviation; the calibrated-LiRA and live-agent results are **3-seed** (each LiRA point aggregates 48 shadow releases, so its own sampling error is small, but we flag the seed count rather than let the reader assume five). The harness and grid are released (§A).

2 Related Work

Federated DP with secure aggregation and discrete noise.

Federated learning [13] trains on decentralised data without centralising it, and SecAgg [8] lets a server compute a sum of client vectors without seeing any summand. To obtain a DP guarantee that composes under that sum without a trusted curator, discrete mechanisms are used: the **distributed discrete Gaussian** [14] and the **Skellam mechanism** [9], the latter being closed under addition (the sum of Skellams is Skellam) and efficient at low bit-width. Prior deployments target **gradient** payloads for model training [15]. We reuse this path *verbatim* but change the payload to **agent-memory embeddings** and the objective to **retrieval**, not learning.

Privacy of LLM-agent memory.

Agent frameworks A-MEM [1] and Mem0 [2] persist distilled per-user memories. Their privacy is under active attack: **MEXTRA** [3] extracts memory content and **MRM-MIA** [4] runs membership inference; a memory-security survey [12] catalogues this attack literature. Defenses so far are **access control** (*Collaborative Memory* [5], which also introduces a shared-vs-private memory split but governs *who reads* a note, not the aggregate) and **on-device masking / redaction** (*MemPrivacy* [6]); neither gives an aggregate formal guarantee. We therefore **cite and reuse the attacks as our evaluation harness** rather than claiming them, and provide the missing mechanism-side guarantee.

Federated retrieval / datastores (the nearest prior art).

DP Datastore Generation [7] is the closest existing mechanism: it partitions data with **locality-sensitive hashing** and adds **calibrated DP noise to each bucket's aggregate**, releasing a private datastore on which membership-inference accuracy falls to near-chance ($\approx 53.6\%$ at $\varepsilon = 5$) — the same LSH-bucket-plus-DP skeleton and privacy-endpoint framing we adopt. It is, however, **centralized and single-curator**, uses **generic additive (continuous) noise with no secure aggregation**, and its payload is a **classification/retrieval datastore**, not cross-user agent memory. Our delta is therefore fourfold: (i) a **federated, curator-free** release under **SecAgg**, where the noise is added distributively and no party ever sees a summand; (ii) the **Skellam** mechanism, whose **closure under summation** is precisely what makes the distributed release *equal* the intended central mechanism (a discrete-Gaussian/continuous-noise datastore does not compose this way under modular SecAgg); (iii) the **agent-memory embedding** payload and cosine-retrieval objective; and (iv) the coupled **dimension/density crossover** and the distilled-payload finding (§7.4, §7.6). **POPri** [10] is federated and private but produces **DP synthetic data via preference optimisation**, not an aggregated memory/preference pool, so it is orthogonal; **Fed-SE** [11] federates agent *self-evolution* under privacy constraints — adapting behaviour, not releasing a shared memory. To our knowledge no prior work aggregates cross-user agent memory under SecAgg + discrete-DP with a measured attack-drop endpoint.

Dimension dependence of DP.

That utility degrades with dimension under DP is classical [16] and was sharpened for deep models by Chen et al. [17]. We do **not** claim the dimension dependence itself. An earlier draft of this work claimed the *coupled* crossover — that in agent memory, dimension governs pre-DP *leakage* and DP *utility-retention* simultaneously, making tiny-d a joint optimum. §7.4 retracts that claim and shows the coupling was an artifact of a data-dependent projection.

3 Threat Model and Problem Setup

Parties.

N users, each with a local agent holding a set of memory notes; an honest-but-curious aggregation server; and downstream agents that query the shared memory.

Goal.

Produce a single **shared memory pool** (a set of K centroid embeddings) that any agent can query by cosine similarity to retrieve relevant knowledge contributed by the fleet, such that the pool carries a **central** (ε, δ)**-DP guarantee at user granularity** and no individual user's notes can be reconstructed or their membership inferred.

Trust / adversary.

No trusted curator: under SecAgg the server observes only the masked sum, so the DP noise need not be added by a trusted party. The adversary is a recipient of the released pool (the server, or any downstream agent). It mounts:

- **Membership inference** (MRMMIA analog): given a candidate note embedding x, decide whether the user who owns x contributed to the pool. Score $s(x) = \cos(x, \mu[\text{bucket}(x)])$; members pulled their own centroid and score higher. Metric: ROC-AUC (0.5 = no leakage).
- **Extraction** (MEXTRA analog): advantage in reconstructing an in-bucket member embedding from the centroid; we report the member/non-member **extraction gap**.

The vulnerable subpopulation — and why it is a lens, not a bound.

It is natural to reason that in a *sum* mechanism leakage must concentrate where **few users contribute to a bucket**, since averaging ought to protect well-populated buckets. We therefore additionally *stratify* the similarity-proxy metrics by the **low-count tail** (buckets with ≤ 3 contributors), as the intuitive worst case and the group agent-memory privacy most obviously cares about (rare/unique notes).

That intuition is, however, **only true of a weak adversary**, and we flag it here because it shaped an earlier version of this work. Averaging dilutes a member's contribution relative to a *population* baseline, which is all an uncalibrated cosine score can measure; it does not remove the member's influence on the centroid, which is what a *per-target* likelihood ratio measures. Our headline attack (the calibrated LiRA of §7.2) is accordingly scored **over all buckets, not the tail**, and it re-identifies members of *dense* coarse-K buckets that the proxy reports as being at chance (§7.1, §7.5). The tail stratification is thus a property of our *evaluation lens*, retained for continuity with the published attacks and for the contrast it draws — it is not a claim about where leakage lives, and it is not a bound on where we looked.

Threat boundary (what we do not defend).

Our adversary is **passive**: a recipient of the released pool — the honest-but-curious server, or any downstream agent — and, under SecAgg, an observer of masked traffic. Two adversaries are explicitly out of scope. (i) **Active, colluding clients.** A malicious participant can inject structured notes to steer particular bucket centroids across

epochs, and query the pool before and after a target user joins, turning the offline LiRA of §7.2 into an *adaptive online* attack against specific delta vectors. Robust aggregation against poisoning of a private vector-retrieval pool is an open problem, orthogonal to the release mechanism analysed here. (ii) **Participation metadata.** SecAgg hides a client's *contents*, not the fact that it participated; §5 specifies mandatory participation with padded dummy payloads to close that channel, an assumption we state but do not measure.

Utility.

For a query q, retrieval routes to the top-k centroids by cosine; utility is whether the *right* content is retrieved (evidence-recall / answer-recall on real benchmarks, topic-accuracy on synthetic corpora).

4 Bucketed Centroid Memory

Let each note i of user u have embedding $e_i \in \mathbb{R}^D$ from a sentence encoder. We reduce to a working dimension d with a **public-seed Gaussian random projection** $P \in \mathbb{R}^{D\times d}$, $P_{jk} \sim \mathcal{N}(0, 1/d)$, L2-normalize, and assign to one of K buckets by random-hyperplane LSH: fix K random anchors $a_1, \ldots, a_K$ and set $\text{bucket}(i) = \arg\max_k \langle e_i, a_k\rangle$. Both P and the anchors are drawn from a **published seed and never touch user data**, so shipping them to clients releases nothing and the ε of §5.4 accounts for the *entire* pipeline. This is not a detail: a PCA fit on the users' notes — the obvious choice, and the one an earlier draft made — is a data-dependent release whose privacy cost is unaccounted, and §7.4 shows it also manufactures a spurious finding.

Each user forms, per bucket k, a **sum-vector** $v_u[k] = \sum_{i:\text{bucket}(i)=k} e_i$ and a **count** $c_u[k]$. The (non-private) shared centroid is

$$\mu[k] \;=\; \text{normalize}\!\left(\frac{\sum_u v_u[k]}{\sum_u c_u[k]}\right). \tag{1}$$

Retrieval for query q returns the top-k buckets by $\cos(q, \mu[k])$. K controls memory *fidelity* (more buckets = finer memory), d controls embedding *resolution*; both, as we show, trade off against privacy.

5 Private Release: SecAgg + Skellam

5.1 Clipping, quantisation, and sensitivity

Per-user clipping and quantisation.

A user's contribution to the release is not one bucket but the *whole* payload: the concatenation $V_u = [v_u[1]; \ldots; v_u[K]] \in \mathbb{R}^{K\cdot d}$. We therefore clip V_u **globally**, to a single L2 bound C, and only then quantise to integers on a fixed grid of resolution $s = \texttt{range\_max}/B$ ($\texttt{range\_max} = 10^6$, B the quantiser bound). This yields integer vectors amenable to SecAgg and to a discrete noise mechanism, with **user-level L2 sensitivity exactly** $\Delta_2 = C \cdot s$.

Neighbouring relation.

Throughout, two datasets are neighbours if one is obtained from the other by **adding or removing a single user** (unbounded DP). This is what makes $\Delta_2 = Cs$ exact: deleting a user removes one clipped payload of norm $\leq C$. (Under the *replace-one* convention the same mechanism would have $\Delta_2 = 2Cs$, and every ε below would double.) We use the same relation for the clip-selection mechanism of §5.2, so the two costs compose against a single definition.

5.2 The clip bound is a mechanism parameter, so it must be public too

C and B are shipped to every client. The natural choice — C = the 95th percentile of the observed payload norms, B = the largest observed coordinate — is a **function of the private corpus**, so releasing it is an unaccounted release, exactly the error §7.4 diagnoses for the projection. An earlier draft of this paper made both. We fix both, and neither is expensive.

- $B := C$ **is free.** Since every note is L2-unit and the payload is clipped first, $|V_u[j]| \leq \|V_u\|_2 \leq C$ for every coordinate j. So $B := C$ is a *public* bound that provably never clips a coordinate, and ε is invariant to B anyway: $\Delta_2/\sqrt{\mu} = 1/\sigma$ regardless of $s = \texttt{range\_max}/B$. Nothing is paid.
- C **is DP-selected.** We choose C from a public geometric grid with the **exponential mechanism** on the rank utility $u(c) = -\,|\#\{u : \|V_u\|_2 \leq c\} - qN|$, $q = 0.95$, sampling c with probability $\propto \exp(\varepsilon_c\, u(c)/2)$. Its sensitivity under **add/remove** deserves care, because the count *and* the target qN both move when a user leaves. Removing a user whose norm is $\leq c$ drops the count by 1 and the target by q, so the argument of $|\cdot|$ shifts by only $1 - q = 0.05$; removing a user whose norm is $> c$ leaves the count alone and shifts the target by q. Hence

$$\Delta u \;=\; \max(1-q,\; q) \;=\; q \;=\; 0.95 \;\leq\; 1, \tag{2}$$

  and the sampler above — which would be ε_c-DP at $\Delta u = 1$ — is in fact $(q\,\varepsilon_c)$-DP. We **charge the full** ε_c, a 5% conservative margin. It composes with the release in RDP via pure-DP $\to$ zCDP: an ε_c-DP mechanism is $(\varepsilon_c^2/2)$-zCDP, hence $\mathrm{RDP}_\alpha \leq \alpha\varepsilon_c^2/2$ [18]. (Had we instead assumed the replace-one convention here while using $\Delta_2 = Cs$ for the release, the two mechanisms would have been accounted against different neighbouring relations — a subtle way to under-count.)

We spend $\varepsilon_c = 0.1$, which costs **0.2% of the budget** (ε 9.29 $\to$ 9.31 at $\sigma = 0.606$; §7.7). The grid is deliberately **coarse** (16 points spanning $[2, 64]$, a public range: a user with n_u unit-norm notes has $\|V_u\|_2 \in [\sqrt{n_u},\, n_u]$, and agent memories hold tens of notes per user), because the exponential mechanism's rank error grows like $\log|\text{grid}|/\varepsilon_c$ [19].

The selection is conservative, and one-sidedly so.

The rank utility saturates at $-0.05N$ for *any* c above the largest observed norm, so at small ε_c the entire upper tail of the grid stays competitive and C is biased **upward**: on

LongMemEval-oracle at $K = 32$ it lands at 21.7 ± 8.5 against a true p95 of 13.7 ± 1.3 (mean ratio 1.6×). That error only ever *adds* noise — it can never under-noise the release — so the guarantee is safe, and the price is utility: evidence-recall@5 at $\varepsilon \approx 9.3$ falls from 0.416 (leaky empirical p95) to **0.406** at $K = 32$, and from 0.117 to 0.100 at $K = 256$, where the signal-to-noise is thinner. We regard a 2% headline utility cost as the correct price for an accounted guarantee, and note the knob: $\varepsilon_c = 0.25$ tightens C to 13.4 ± 1.3 and recovers most of the loss, at a total ε of 9.41 instead of 9.31.

Remark 1 (Cross-bucket sensitivity: why the clip must be global) Clipping each bucket-vector *separately* to C would **not** yield sensitivity C. A user with notes in b_u distinct buckets would contribute up to C in each, so the payload sensitivity would be $\sqrt{\sum_k \|v_u[k]\|^2} \le \sqrt{b_u} \cdot C$, and an accountant still charging $\Delta_2 = Cs$ would under-count by $\sqrt{b_u}$. The blow-up is governed by $b_u \le \min(n_u, K)$, the number of buckets the user *touches* — **not by K alone**, since a user with n_u notes cannot occupy more than n_u buckets. On LongMemEval-oracle (mean 21.9 notes/user, max 72) the busiest user touches $b_{\max} = 21$ buckets at $K = 32$ and 54 at $K = 1024$, so per-bucket clipping would have inflated the true budget from the reported $\varepsilon \approx 9.31$ to $\varepsilon \approx 69$ ($K = 32$) and $\varepsilon \approx 159$ ($K = 1024$) — a broken guarantee. Our implementation clips globally (`clip_rows_to_norm(V_u.reshape(1,-1), C)` in `scripts/agentmem/_agentmem_leakage.py`), so every ε reported in this paper is sound; line 8 of Algorithm 1 makes this explicit.

Skellam noise, composed under the sum.

Each user adds Skellam noise (a difference of two Poissons, per-coordinate variance $\mu = (\sigma Cs)^2$) to their quantised vector and submits it through **SecAgg**, so the server recovers only $\sum_u (\text{quantise}(\text{clip}(v_u)) + \text{Skellam})$. Because the sum of independent Skellams is Skellam, the *released sum* carries the target noise with **no trusted curator**. Dequantising and dividing by the (also-released or cancelled, §5.3) count gives the private centroid $\tilde{\mu}$.

Implementation constraints of the integer path.

Three details the accountant assumes and an implementation must honour.

- *Modular field size.* SecAgg sums in a finite field $\mathbb{Z}_p$; too small a p and the integer sum wraps, silently corrupting the aggregate. Per coordinate the signal is bounded by $N \cdot$ `range_max` and the aggregate noise has standard deviation σCs, so it suffices that $p > 2\,(N \cdot \texttt{range\_max} + \kappa\,\sigma Cs)$ for a tail factor κ (we use $\kappa = 6$). At our operating point ($N = 500$, `range_max` $= 10^6$, $\sigma Cs = 8.4 \times 10^5$) this is $p > 10^9$. The reference implementation aggregates in `int64`, exceeding the bound by nine orders of magnitude; the measured peak coordinate of the noised aggregate is 1.8×10^7. (The $\sqrt{d}$ that one might expect here does not appear: the constraint is per-coordinate.)
- *Mandatory participation and padding.* A client with no new notes must still submit. It forms the all-zero payload, adds its Skellam share (line 11) and its SecAgg mask, and submits a vector of identical shape. Because the noise is injected *before* masking, an idle client's submission is distributionally indistinguishable from an active one's, so neither the server nor a network observer learns who used their agent this epoch.

Uniform payload shape likewise prevents inferring that a user's memory is narrow (few occupied buckets) from the ciphertext.

- *Rounding, and what clipping does to heavy users.* round(vs) can lift the integer norm above Cs by at most $\sqrt{Kd}/2$: at our operating point 16 against $\Delta_2 = 1.39 \times 10^6$, a 10^{-5} relative inflation, which we absorb rather than adopt the conditional randomised rounding of [14]. Note also that the clip of line 8 is a **scalar rescale**: it shrinks a heavy user's magnitude but leaves the direction of V_u exactly unchanged, so a high-activity user's semantics are *down-weighted, never distorted*. A deployment that wishes to bound that down-weighting can cap notes-per-user before aggregation, which lowers C and hence the absolute noise.

5.3 Vector-only release, and what the count channel is (and is not) for

The natural design releases **two** channels (the sum-vector and the counts), costing two RDP compositions. For the *direction* of the retrieval centroid the count is redundant: dividing by the scalar count and L2-normalising cancels it, normalize(sv/sc) = normalize(sv). The count therefore never affects the ranking *among the buckets we keep*, and releasing its magnitude is strictly wasteful. We release **only the summed vector** (one composition): identical retrieval to the last decimal at $\sim$1.5$\times$ **tighter** ε (§7.7). We call this the *vector-only* release.

What the count channel was silently doing: occupancy.

Normalisation is not innocent for a bucket nobody filled. If bucket k has no contributors then $S[k]$ is *pure Skellam noise*, and line 18 of Algorithm 1 projects it onto the unit sphere — manufacturing a random unit vector that competes for top-k slots against genuine centroids. Cosine ranking cannot tell the two apart, so a release that keeps empty buckets can be flooded with false positives. **Count magnitude cancels; occupancy does not.** Three facts make this precise (all measured in §7.11):

1. **It does not arise at the operating points.** With 10,957 notes over 500 users, **no bucket is empty for** $K \leq 128$ **on any seed**, and $0.23 \pm 0.19\%$ at $K = 256$ (at most one bucket of 256). Every retrieval number in this paper is reported at $K \leq 256$, and the recommended point is $K = 32$. The noisy-normalisation failure mode is therefore **absent from the reported utility**, not tuned away.
2. **Where it does arise, occupancy is expensive to release.** Under **user-level** DP a count channel is *not* the cheap sensitivity-1 object it would be under note-level DP: deleting a user erases *all* of its entries at once. Even a **binary user-indicator** channel — the cheapest sensible variant, since a user contributes 0/1 per bucket — has L2 sensitivity $\sqrt{b_u}$ ($\approx$ 5–8 here, and its DP-selected public bound lands at 9.7–12.4), **not 1**. The indicator channel is nonetheless the right one: at a full second composition ($\sigma_c = \sigma_v$, ε 9.31 $\rightarrow$ 13.94) it detects empty-vs-occupied at AUC 0.87/0.76/0.70 for K = 512/1024/2048, against 0.82/0.68/0.63 for raw counts. Made cheap ($\sigma_c = 2$, $\varepsilon = 9.78$) it falls to 0.72/0.59/0.55. Thresholding the released vector's own norm $\|S[k]\|$ against the analytic noise floor $\sigma C\sqrt{d}$ is **free** (post-processing) but is at **chance**: AUC 0.50/0.49/0.48.

3. **The difficulty is intrinsic, not an implementation shortfall.** At the densities where buckets are near-empty, the noise that drives membership inference to chance in a one-contributor bucket (§7.5) is *the same noise* that hides whether the bucket has a contributor at all. **Occupancy and membership are the same signal**; no mechanism hides the member and reveals the bucket.

We therefore state the claim precisely: **vector-only dominates the two-channel design at a fixed occupancy rule**, and at the recommended K that rule is vacuous because no bucket is empty. The sound response to a high-K deployment is to *increase density* (coarsen K, which also maximises retention, §7.3), not to bolt on a count channel that cannot pay for itself.

No occupancy oracle in the reported numbers. For the leakage measurement (§7.5, §7.6, §7.9) we release the **vector-only** memory with *no* occupancy suppression: $\tilde{\mu}[k] = \text{normalize}(S[k])$ for every bucket, so an empty bucket carries normalized Skellam noise and nothing reads the clean counts. This is the recommended mechanism of this section, and it removes the non-private gate an earlier draft used. It does not change the conclusion — dropping the gate moves every tail-AUC by ≤ 0.003 (5-seed), inside the noise, because a member always occupies its own bucket and a non-member in an empty bucket scores against noise either way — but it means those tables contain no oracle step. The only clean-count quantity that survives is the **tail *stratification*** (≤ 3 contributors), which is a property of our *evaluation lens*, not of the release: the adversary never sees it, and it selects which rows of results we report rather than what the mechanism emits. The occupancy *rule* a dense high-K deployment would need for retrieval is a separate question, priced in §7.11.

5.4 Privacy accounting (exact Skellam-RDP)

We account with the exact discrete guarantee of Agarwal, Kairouz & Liu (NeurIPS 2021, Thm. 3.5) [9], converting to (ε, δ) through Rényi DP [20]:

$$\varepsilon_{\text{RDP}}(\alpha) \;\leq\; \frac{\alpha\Delta_2^2}{2\mu} \;+\; \min\left\{ \frac{(2\alpha-1)\Delta_2^2 + 6\Delta_1}{4\mu^2},\; \frac{3\Delta_1}{2\mu} \right\}, \tag{3}$$

with μ the per-coordinate released variance, $\Delta_2 = Cs$, and $\Delta_1 \leq \sqrt{d} \cdot \Delta_2$. Composition over releases is additive in RDP; we convert to (ε, δ) by $\varepsilon = \min_\alpha [\text{RDP}(\alpha) + \ln(1/\delta)/(\alpha - 1)]$ ($\delta = 10^{-5}$). This is implemented as `skellam_rdp_epsilon(...)` in `qpriviot_fl/privacy_utils.py` and replaces the approximate single-shot Gaussian labels used during exploration. **Every ε in this paper is the rigorous Skellam-RDP value.** Table 1 gives a representative mapping at the $K = 32$, $d = 32$ operating point.

The exploration labels "$\varepsilon = 8$ / $\varepsilon = 3$" correspond to the rigorous $\varepsilon \approx 9.3$ / $\varepsilon \approx 3.2$ under the recommended vector-only release, clip selection included; we use those rigorous values throughout, in the tables and on every figure axis. Because all utility and leakage effects are **monotone in** σ, re-labelling the ε axis leaves every ordering, retention percentage, and AUC-drop unchanged.

On the choice of δ.

The guarantee is at **user** granularity, so the population size that δ must be compared against is the number of **users** ($N = 500$), not the number of notes — the **s**-variant of

Table 1 $\sigma \to \varepsilon$ at the $K = 32$, $d = 32$ operating point ($\delta = 10^{-5}$). Every ε is the *total*: the Skellam release plus the $\varepsilon_c = 0.1$ spent DP-selecting the clip bound (§5.2). The projection and $B := C$ contribute nothing. The "classic" column is the label the exploration grid was run under and is *invalid* for $\varepsilon > 1$; it is shown only to map the run labels onto the rigorous budgets.

	classic label	Skellam-RDP (exact)	
σ	(invalid, $\varepsilon > 1$)	**vector-only (1 rel.)**	two-channel (2 rel.)
0.303	15.99	22.11	33.31
0.606	7.99	**9.30**	13.94
1.615	3.00	**3.21**	4.63
2.854	1.70	**1.81**	2.56

§7.9 has 246,073 *notes* contributed by those same 500 users. The standard requirement is $\delta \ll 1/N$: here $1/N = 2 \times 10^{-3}$, so $\delta = 10^{-5}$ sits **200× below** the threshold, comfortably inside the safe regime. The requirement *tightens* with the user count rather than relaxing, so scale is the stress case, not the reassurance: a production fleet of $N = 10^5$–10^6 **users** would need $\delta \ll 10^{-5}$. Our accountant takes δ as a parameter, and the cost of retightening is mild — at fixed σ, $\varepsilon \approx 9.30 \to 10.07/10.84/11.44$ for $\delta = 10^{-6}/10^{-7}/10^{-8}$ (and $\varepsilon \approx 3.21 \to 3.50/3.76/4.01$). Three orders of magnitude in δ cost $< 23\%$ **in** ε, so the mechanism carries to production-scale δ without re-tuning.

5.5 The full mechanism

We consolidate §4–§5.3 into a single end-to-end procedure (Algorithm 1). The **shared public parameters** are the random-hyperplane LSH anchors $\{a_1, \ldots, a_K\}$, the public-seed random projection $P : \mathbb{R}^D \to \mathbb{R}^d$, the clip bound C, the quantiser bound B and step $s = \texttt{range\_max}/B$, the noise scale σ, a **survival floor** $M_{\min}$ (§5.6), and the clip-selection budget ε_c (§5.2; the clip bound C and the quantiser bound $B := C$ are derived once, publicly, from it). The target per-coordinate *aggregate* Skellam variance is $\mu = (\sigma C s)^2$; under SecAgg each client injects a $1/M_{\min}$ share, so the masked integer sum realises at least μ with **no trusted curator** and **no summand in the clear**. Only the **sum-vector channel** is released (§5.3): dividing by the count and L2-normalising cancels the count.

Why each step matters.

Lines 3–5 place a note in the *shared* coordinate frame so different users' notes land in comparable buckets; both P and the anchors come from a public seed, so this step is data-independent and free (§4, §7.4). Lines 6–8 are the DP sensitivity: the clip is applied **once, to the concatenated payload**, because a user who touches b_u buckets would otherwise contribute C to each and the true sensitivity would be $\sqrt{b_u}\,C$ (Remark 1). Lines 1–2 fix the two public bounds before any user data is touched in the clear: C by the exponential mechanism (cost ε_c, 0.2% of the budget) and $B := C$ for free. The quantise/noise lines move to the integer domain and add the discrete noise *there*, which is what makes it survive modular SecAgg. The mask line hides

Algorithm 1 Private Federated Memory Aggregation (SecAgg + Skellam, vector-only release). Public/shared parameters are as listed in the text above.

Server, once *(public parameters; §5.2)*

1: $C \leftarrow \textsc{ExpMech}_{\varepsilon_c}\big(\text{grid},\ u(c) = -|\#\{u : \|V_u\|_2 \le c\} - 0.95N|\big)$ ▷ DP-selected clip
2: $B := C; \quad s \leftarrow \texttt{range\_max}/B; \quad \mu \leftarrow (\sigma C s)^2$ ▷ $|\text{coord}| \le \|V_u\|_2 \le C$

Client u *(local; note text never leaves the device)*

3: **for** each note i of user u **do**
4: $\quad e_i \leftarrow \text{normalize}\big(P(\text{Encoder}(\text{note}_i))\big)$ ▷ 384-d $\to d$, L2-unit
5: $\quad b(i) \leftarrow \arg\max_k \langle e_i, a_k \rangle$ ▷ LSH bucket
6: $v_u[k] \leftarrow \sum_{i:b(i)=k} e_i \quad \text{for } k = 1 \dots K$ ▷ per-bucket sum-vector
7: $V_u \leftarrow [\, v_u[1]; \dots; v_u[K]\,] \in \mathbb{R}^{Kd}$ ▷ the user's FULL payload
8: $V_u \leftarrow V_u \cdot \min\big(1,\ C/\|V_u\|_2\big)$ ▷ **global** clip $\Rightarrow \Delta_2 = Cs$ (Rem. 1)
9: **for** each bucket $k = 1 \dots K$ **do**
10: $\quad z_u[k] \leftarrow \text{round}\big(v_u[k] \cdot s\big) \in \mathbb{Z}^d$ ▷ quantise; $B := C$ never clips
11: $\quad \tilde{n}_u[k] \leftarrow \text{Skellam}(\mu/M_{\min})$ ▷ noise share, sized to survivors
12: $\quad \tilde{z}_u[k] \leftarrow z_u[k] + \tilde{n}_u[k]$ ▷ DP noise, in the integer domain
13: $\quad m_u[k] \leftarrow \tilde{z}_u[k] + \text{mask}_u[k], \quad \sum_u \text{mask}_u[k] = 0$ ▷ SecAgg zero-sum mask
14: **submit** $\{m_u[k]\}_{k=1}^K$ to the server

Server *(honest-but-curious; sees only masked sums)*

15: **if** fewer than $M_{\min}$ clients survive **then abort**
16: **for** each bucket $k = 1 \dots K$ **do**
17: $\quad S[k] \leftarrow \sum_{u \text{ alive}} m_u[k] = \sum_u z_u[k] + \text{Skellam}\big(\frac{M}{M_{\min}}\mu\big)$ ▷ masks cancel
18: $\quad \tilde{\mu}[k] \leftarrow \text{normalize}\big(S[k]/s\big)$ ▷ **every** bucket; no occupancy oracle (§5.3)
19: **release** the private centroid memory $\{\tilde{\mu}[k]\}_{k=1}^K$

Retrieval *(any downstream agent, query q)*

20: **return** top-k buckets by $\cos\big(\text{normalize}(P(\text{Encoder}(q))),\ \tilde{\mu}[k]\big)$ over $\{k : \tilde{\mu}[k] \neq 0\}$

Accounting *(§5.4, exact Skellam-RDP)*

21: $\Delta_2 \leftarrow Cs; \quad \Delta_1 \leftarrow \sqrt{d}\,\Delta_2$ ▷ valid *because* line 8 clips globally
22: $\varepsilon_{\text{RDP}}(\alpha) = \frac{\alpha\Delta_2^2}{2\mu} + \min\big\{\frac{(2\alpha-1)\Delta_2^2+6\Delta_1}{4\mu^2},\ \frac{3\Delta_1}{2\mu}\big\} + \frac{\alpha\varepsilon_c^2}{2}$ ▷ last term: DP clip selection
23: $\varepsilon \leftarrow \min_\alpha \big[\varepsilon_{\text{RDP}}(\alpha) + \ln(1/\delta)/(\alpha-1)\big]$ ▷ 1 release $\Rightarrow \sim 1.5\times$ tighter ε

the contribution, and the server's sum shows the masks cancel so the server learns only the sum. That sum is the crux: because the sum of independent Skellams is Skellam, the per-client $\mu/M_{\min}$ shares **compose to at least the central target** μ, so the distributed release *equals* (or, on good turnout, exceeds) the intended central mechanism — a property a continuous-Gaussian datastore lacks under modular arithmetic. Line 18 releases **every** bucket, including empty ones: an empty bucket therefore emits normalized Skellam noise rather than being suppressed. That is deliberate and it is the reason the reported ε is honest. *Any* occupancy gate that consults the true counts — the obvious implementation, and the one an earlier version of our own harness used — is a non-private read of the corpus and an **unaccounted release**, precisely the error §5.2 and §7.4 diagnose elsewhere. We therefore pay its cost in utility instead of hiding it in the mechanism: every retrieval number in this paper is measured with *no* gate (`--gate none`), which at $K \le 256$ changes nothing (no bucket is empty) and at $K \ge 512$ costs the one or two points of retention visible in Table 4. §7.11 prices what a *private* gate would cost, and shows it cannot pay for itself. The implementation is `apply_distributed_skellam_noise` / `skellam_noise` / `generate_zero_sum_masks` / `quantize` in `qpriviot_fl/privacy_utils.py`, and the accounting is `skellam_rdp_epsilon(...)`.

Table 2 Client dropout silently breaks the guarantee. True ε realised when each of N clients injects a naive μ/N noise share but only M survive.

survivors M/N	1.00	0.90	0.70	0.50	0.25
true ε (nominal $\varepsilon \approx 9.30$)	9.30	9.91	11.61	**13.94**	**22.11**
true ε (nominal $\varepsilon \approx 3.21$)	3.21	3.39	3.87	**4.63**	**6.74**

5.6 Client dropout: the noise floor must be sized to survivors, not to N

Line 11 injects a $1/M_{\min}$ share rather than the naive $1/N$, and the difference is not cosmetic. SecAgg deployments lose clients mid-protocol. If each of N clients injects Skellam(μ/N) and only $M < N$ survive to reconstruction, the server recovers aggregate variance $\mu_{\text{actual}} = (M/N)\,\mu$ — an **effective noise multiplier** $\sigma_{\text{eff}} = \sigma\sqrt{M/N}$. The guarantee then degrades silently, and worst exactly when participation is worst (Table 2).

Two sound remedies, either of which restores a valid guarantee:

- **Calibrate to a survival floor** $M_{\min}$ (Algorithm 1, line 11). Each client injects Skellam$(\mu/M_{\min})$, so any turnout $M \geq M_{\min}$ realises variance $(M/M_{\min})\,\mu \geq \mu$: the guarantee **can only improve**, never degrade, and the server **aborts** rather than releasing if $M < M_{\min}$. The price is over-noising on good turnout. With $M_{\min} = N/2$ calibrated to $\varepsilon \approx 9.30$ *at the floor*, a full-turnout round realises $\sigma = 0.857$ and $\varepsilon \approx 6.31$ — utility is paid for at the floor, privacy is banked at the ceiling.
- **Fault-tolerant distributed noise generation.** Secret-share each client's noise seed exactly as SecAgg already secret-shares its mask seeds [8], so surviving clients reconstruct the dropped clients' noise shares and the aggregate variance is exactly μ for any M. This removes the utility penalty at the cost of another reconstruction round.

Our experiments assume full participation ($M = N = 500$, i.e. $M_{\min} = N$), with no straggler model. Dropout robustness is therefore something this paper *specifies* but does not *measure*; we record it as a limitation (§8), not a claim.

5.7 Repeated release: memory is re-aggregated, so the budget must compose

A model is trained once; a **memory** is not. Users write notes continuously, and a deployed pool is re-aggregated on some cadence — which means the release of §5.5 happens T times, not once, and **the guarantee that matters is the composed one**. This is the assumption most easily left implicit, and leaving it implicit is not benign: RDP composes *additively*, so re-releasing at the single-shot $\sigma = 0.606$ costs the budgets in the last column of Table 3 — $\varepsilon \approx 93$ for a month of daily rounds, $\varepsilon \approx 153$ for a year of weekly ones. A mechanism advertised at $\varepsilon \approx 9.3$ and re-run nightly is not an $\varepsilon \approx 9.3$ mechanism.

Table 3 Buying a re-aggregation cadence inside a **fixed** total budget ($\varepsilon = 9.31$, $K = 32$, $d = 32$, $\delta = 10^{-5}$; clip bound selected once, publicly). σ is solved from the Skellam-RDP accountant at T releases; retention is measured at that σ (5 seeds), not extrapolated. The last column is what those T releases would *actually* have cost had we kept the single-shot $\sigma = 0.606$ and simply re-released — i.e. the budget an unaccounted deployment would silently be spending.

T	cadence	σ needed	recall@5	Retention	naive ε
1	single shot (this paper)	0.606	0.406	**95%**	9.3
4	quarterly, 1 year	1.211	0.382	89%	22.1
12	**monthly, 1 year**	2.098	0.349	**81%**	44.2
30	daily, 1 month	3.317	0.313	73%	93.3
52	weekly, 1 year	4.367	0.283	66%	153.3

The remedy is not to release once and hope. Because the composed RDP is linear in T while the per-release RDP falls as $1/\sigma^2$, the noise scale needed to hold a **fixed total budget** grows only as $\sqrt{T}$, so a cadence can simply be *bought* up front: pick the T the deployment needs, solve for σ, and pay for it once in utility. Table 3 prices that trade at the recommended $K = 32$, reading retention off runs actually executed at each σ rather than extrapolating.

The headline is that the mechanism **survives a realistic cadence**: a pool re-aggregated **monthly for a full year** — twelve releases, each incorporating every note written since the last — costs **81% retention at the same** $\varepsilon \approx 9.3$ we report throughout, against 95% for the single shot. Even a month of *daily* re-aggregation holds 73%. What a deployment may **not** do is re-release at the single-shot σ and continue to quote the single-shot ε.

Two properties make this cheaper than it first looks. The **public parameters do not recompose**: the projection and anchors are public-seed (zero ε at any T), and the clip bound C is a *public* bound over the payload norm, so it is selected once and reused — we charge ε_c a single time, not T times, and re-selecting it per round would be an avoidable waste. And the guarantee is **user-level and unbounded-DP**, so a user who writes no notes between rounds contributes an all-zero payload and their privacy is not re-spent by the calendar; it is the *release* that composes, not the user. Sizing σ to the planned horizon and aborting past it is the sound discipline, exactly as $M_{\min}$ is for turnout (§5.6).

6 Experimental Setup

Datasets / payloads.

- **LongMemEval (oracle)** [21]: real long-horizon conversational memory. Users = question haystacks, notes = dialogue turns, queries = questions, ground truth = evidence turns. The loader yields **500 users, 10,957 note embeddings, 479 queries with evidence**.

- **LongMemEval (s)** [21]: the full-haystack variant, same 500 users but **246,073 note embeddings** (∼96% distractors), ∼22× the oracle corpus, used for the at-scale generality check (§7.9).
- **LLM-distilled notes** (A-MEM/Mem0-style): dialogue turns distilled into memory notes by a local LLM (Ollama `qwen2.5:7b`). **500 users → 6,116 distilled notes**; a 100-user pilot → 1,715 notes. This is the realistic agent-memory payload, and the memory store for the live-agent attacks of §7.10.
- **Synthetic / real-embedding controls**: 20-Newsgroups with TF-IDF→SVD and with real `all-MiniLM-L6-v2` embeddings, for controlled $N/K/d$ sweeps and a clean topic-accuracy metric.

Embeddings.

`all-MiniLM-L6-v2` (384-d) [22], reduced to the working dimension d by a **public-seed Gaussian random projection** (data-independent; §4). Buckets are K random-hyperplane anchors from the same public seed. For the §7.4 ablation only, we additionally run two alternatives: `data-PCA` (PCA fit on the users' notes — the unaccounted release) and `publicPCA` (PCA fit on a disjoint public corpus: 20-Newsgroups for the LongMemEval runs, LongMemEval for the 20NG runs). At $d = 384$ the projection is the identity, which is the ablation's control. Selected by `--proj` {`randproj,pca,publicpca`}.

Mechanism.

Faithful crypto path (`privacy_utils` quantize → Skellam → SecAgg-sum → dequantize), `range_max` $= 10^6$. Vector-only release unless stated. The clip bound C is DP-selected once per run with the exponential mechanism at $\varepsilon_c = 0.1$ (§5.2, `--clip-eps`), and the quantiser bound is $B := C$; both are public parameters of the released mechanism and their cost is inside every ε we report.

Metrics.

- *Utility*: evidence-recall@5 (LongMemEval), answer-recall@5 (distilled), topic-accuracy (controls); chance ≈ topk/K. We report **retention@**ε = utility(ε)/utility(clean).
- *Leakage*: membership-inference ROC-AUC and **low-FPR TPR** (calibrated LiRA, §7.2) over all buckets and the **low-count tail** (≤ 3 contributors), a reconstruction-decode extraction rate (§7.2), and, against a live agent (§7.10), MEXTRA verbatim-recovery and MRMMIA-AUC. AUC 0.5 / TPR ≈ FPR / chance-level decode = no leakage.

Protocol.

5 seeds (0–4), **final metric** (not peak), mean ± std; the calibrated LiRA of §7.2 uses 3 seeds × 48 shadow releases and the live-agent attacks of §7.10 run on `qwen2.5:7b`. Members vs non-members are a same-distribution split of the note pool (aggregated vs held-out), avoiding any train/test confound.

Table 4 The joint frontier: retrieval utility and *calibrated* leakage at the **same** K (LongMemEval-oracle, $d = 32$, vector-only; utility 5-seed, LiRA 3-seed × 48 shadows). Leakage is the offline-LiRA of §7.2, not the cosine proxy — which, as §7.5 shows, reports chance at exactly the K one would deploy. TPR@1%FPR floor = 0.010.

		utility (evidence-recall@5)			LiRA AUC		TPR@1%FPR		decode-extract	
K	Chance	Clean	$\varepsilon \approx 9.3$	Reten.	clean	ε9.3	clean	ε9.3	clean	ε9.3
32	0.156	0.428 ± 0.046	$\mathbf{0.406 \pm 0.054}$	**95%**	0.833	**0.502**	0.175	**0.011**	0.771	0.396
64	0.078	0.320 ± 0.045	0.276 ± 0.049	86%	0.908	0.505	0.357	0.011	0.667	0.188
128	0.039	0.233 ± 0.031	0.167 ± 0.039	**72%**	0.954	0.502	0.552	**0.011**	0.794	**0.070**
256	0.020	0.167 ± 0.022	0.100 ± 0.035	60%	0.981	0.503	0.750	0.011	0.789	0.029
512	0.010	0.152 ± 0.026	0.062 ± 0.031	41%	0.993	0.505	0.874	0.013	0.874	0.007
1024	0.005	0.135 ± 0.017	0.032 ± 0.016	24%	0.997	0.504	0.945	0.013	0.912	0.003
2048	0.002	0.128 ± 0.013	0.016 ± 0.010	13%	0.998	0.505	0.957	0.012	0.928	0.001

7 Results

The tables below are regenerated directly from the released grid (`experiment_results/rerun_grid_rp/` and `rerun_grid_s_rp/`), so they stay in sync with the artifacts. All numbers are **5-seed means ± std**; ε values are the rigorous Skellam-RDP labels of §5.4, **including the ε spent DP-selecting the clip bound** (§5.2), where the exploration "$\varepsilon = 8$ / $\varepsilon = 3$" $= \varepsilon \approx 9.3$ / 3.2. Every parameter of the released mechanism — the projection, the anchors, the clip bound C, the quantiser bound B — is public or DP-accounted, so these ε cover the whole pipeline.

Seed noise on these corpora is **not** negligible (evidence-recall@5 carries a 5-seed std of ±0.02–0.06), so we report it in every cell and avoid reading differences smaller than it.

7.1 The joint frontier: both axes, one operating point

A privacy/utility claim is a claim about *one release*. It is therefore only meaningful if both axes are measured at the **same** operating point — and this is precisely the discipline our own superseded draft failed. That draft measured utility only at $K \leq 256$ and leakage only at $K \geq 512$: two disjoint grids, from which it quoted "95% retention" (a $K = 32$ number) and "tail-AUC 0.93 → 0.53" (a $K \geq 512$ number) as though they described a single mechanism. They did not. We therefore re-ran the missing cells — utility at $K \geq 512$, and the calibrated attack of §7.2 at $K \leq 256$ — and report the **joint frontier** (Table 4, Fig. 1).

The frontier has an interior optimum, and it is where we already were.

Three readings, all load-bearing.

1. **DP defeats the calibrated attack at *every* K.** LiRA AUC lands in 0.502–0.505 and TPR@1%FPR in 0.011–0.013 — the false-positive floor — for all seven fidelities, even as the *clean* attack climbs from AUC 0.833 to 0.998. The privacy axis is therefore **flat**: it does not trade against K. Choosing K is a pure utility decision, which is what makes the frontier readable.

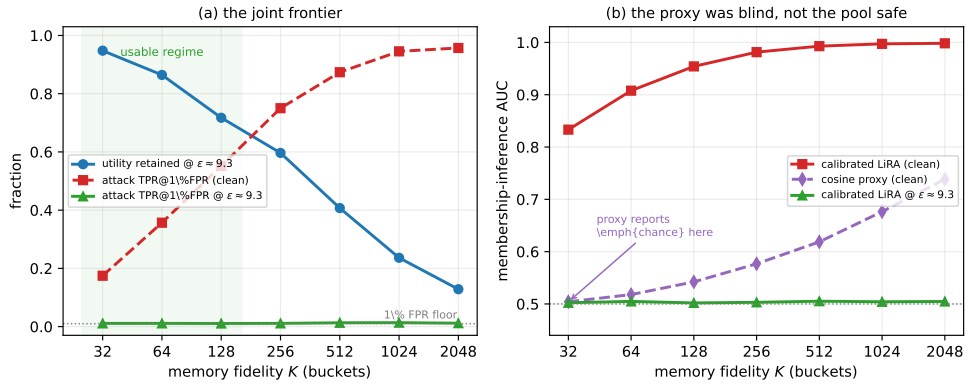

Fig. 1 (a) The joint frontier. Utility retention falls monotonically with fidelity K while clean leakage rises; DP pins the calibrated attack at the 1% false-positive floor *uniformly*, so the frontier is governed by the utility axis alone and the coarse-K regime dominates. **(b) Why the superseded draft missed this.** The uncalibrated cosine proxy (purple) reports *chance* at $K = 32$ on the same clean release where the calibrated LiRA (red) reads AUC 0.833. The proxy was blind; the pool was not safe (§7.5).

2. **So the operating point is set by retention, and coarse wins.** Retention falls monotonically 95 → 86 → 72 → 60 → 41 → 24 → 13% as K grows 32 → 2048, because utility is set by *effective contributors per bucket*: coarse buckets pool more users, so DP is nearly free; fine buckets starve, so noise bites. $K = 32$ **is the recommended point**: 95% retention with the membership attack at the floor.
3. **Membership and reconstruction do not die at the same K, and we say so.** At $K = 32$ the *membership* attack is fully defeated (TPR@1%FPR 0.175 → 0.011) but *reconstruction-decode* only halves (0.771 → 0.396, against a noise floor of ≈0.08 measured at $\varepsilon \approx 1.1$). This is not noise: at coarse K a centroid averages ∼340 notes, so "the top-1 decoded note is a true in-bucket member" remains partly achievable simply because the bucket is large — the attack recovers a note's *region*, not its owner, and the disclosure is membership in a 340-note bucket rather than identification of a user. It is nonetheless above floor, and a deployment that must defeat reconstruction as well should run at $K = 128$, where decode reaches 0.070 (at floor) and membership stays at 0.011, for **72% retention**. We report both points rather than quoting the more flattering one.

The honest summary of the frontier is therefore: *the privacy axis is free across the whole sweep; pick K for the retrieval fidelity you need, and pay the retention it costs.* $K = 32$ if membership is the threat (95%); $K = 128$ if reconstruction is too (72%). What one may **not** do — and what the superseded draft did — is read utility off one end of this table and leakage off the other.

Table 5 Calibrated offline-LiRA and reconstruction-decode extraction (LongMemEval oracle, $d = 32$, 3 seeds $\times$ 48 shadow releases). The clean release is far more leaky than the cosine proxy revealed, and DP still defeats the strong attack on every axis at once.

K	Release	AUC	TPR@1%FPR	TPR@0.1%FPR	Decode-extract
1024	clean	0.997	**0.945**	0.784	0.912
1024	$\varepsilon \approx 9.3$	**0.504**	**0.013**	0.001	0.003
1024	$\varepsilon \approx 3.2$	0.501	0.012	0.002	0.001

Table 6 LiRA across the fidelity sweep. Clean leakage rises with K; DP pins every cell at the false-positive floor.

K	Clean TPR@1%FPR	$\varepsilon \approx 9.3$ TPR@1%FPR	Clean decode	$\varepsilon \approx 9.3$ decode
512	0.874	0.013	0.874	0.007
1024	0.945	0.013	0.912	0.003
2048	0.957	0.012	0.928	0.001

7.2 Calibrated attack: LiRA membership inference and extraction

The similarity scores in §7.4–§7.6 embody the *measurement principle* of MEXTRA/MRMMIA but are a weak, uncalibrated proxy. We now run the **modern MIA standard** against the released pool: an **offline LiRA** that, for each candidate note, estimates its membership-score distribution when it is *aggregated into* vs *held out of* the pool from many **shadow releases** (cheap here: our aggregation is numpy, not model training) and tests with the per-target likelihood ratio. We report **ROC-AUC and the low-FPR TPR** (the operationally meaningful metric that AUC-only proxies hide), plus a **reconstruction-decode extraction** attack (MEXTRA analog): decode each released centroid to its nearest note and score a hit when the top-1 decoded note is a true in-bucket member. Fits use a shadow split disjoint from the evaluation shadows (no train-on-test bias).

Calibration exposes what cosine-AUC missed (Table 5): on the clean pool an adversary re-identifies members at ≈95% **true-positive rate at a 1% false-positive budget** and reconstructs the correct in-bucket note for **91% of centroids** — the untreated shared memory is almost fully de-anonymising. The Skellam release drives the same attack to **chance on every axis at once**: AUC 0.997 → 0.504, TPR@1%FPR 0.945 → 0.013 (the FPR floor), and extraction 0.912 → 0.003. Note that the calibrated attack lands at 0.504 where the uncalibrated cosine proxy of §7.5 still reads 0.53–0.55: the proxy's residual was an artifact of its own weakness, not a real membership signal.

The effect holds across the fidelity sweep (Table 6): clean leakage rises with K exactly as §7.5 predicted, while DP pins every cell at chance.

Table 7 LongMemEval-oracle retrieval utility by bucket fidelity K (evidence-recall@5, $d = 32$, vector-only release, 5-seed mean ± std).

K	Chance	Clean	$\varepsilon \approx 9.3$	$\varepsilon \approx 3.2$	Retention@$\varepsilon \approx 9.3$
32	0.156	0.428 ± 0.046	$\mathbf{0.406 \pm 0.054}$	0.363 ± 0.059	**95%**
64	0.078	0.320 ± 0.045	0.276 ± 0.049	0.217 ± 0.046	86%
128	0.039	0.233 ± 0.031	0.167 ± 0.039	0.124 ± 0.029	72%
256	0.020	0.167 ± 0.022	0.100 ± 0.035	0.057 ± 0.031	60%

This LiRA is scored **per candidate note over all buckets**, not only the sparse tail: the TPR@1%FPR of 0.945 above is the all-bucket figure. It therefore already covers the *dense*-bucket case in which a single distinctive note pulls a centroid otherwise averaged over dozens of predictable "bystander" contributors — the likelihood ratio is computed against shadow releases with and without that exact note, so a unique contribution buried among benign ones is precisely what the test is powered to detect. Sparse-tail stratification (§7.5) reports where leakage *concentrates*; it does not bound where we *looked*.

On **real `all-MiniLM-L6-v2` embeddings** the calibrated attack corroborates §7.4's negative result: under the data-independent projection clean TPR@1%FPR is nearly **flat** across dimension, $0.967 \to 0.979$ for $d = 32 \to 384$ (decode $0.895 \to 0.966$), where the superseded data-PCA grid showed a steep $0.908 \to 0.979$. Tiny-d buys essentially no leakage reduction once the projection stops pre-anonymising the corpus, and DP collapses every cell to chance at $\varepsilon \approx 9.3$ regardless of d (TPR@1%FPR 0.011/0.010). (We headline the low-FPR TPR because AUC becomes an unstable estimator at the largest σ, where it can tick up to ≈ 0.59 even as TPR@1%FPR stays at the ≈1% floor; the attack is defeated operationally regardless.) This **upgrades the endpoint from a proxy to a calibrated attack**, leaving only the *LLM-agent-prompting* form of MEXTRA/MRMMIA — a live agent querying a text store, a different release model than our centroid pool — as the subject of §7.10.

7.3 Utility in detail (LongMemEval oracle)

Retrieval survives DP in the coarse-K regime and degrades gracefully as memory fidelity K grows (Table 7, Fig. 2).

At the recommended operating point ($K = 32$, $d = 32$) the private pool keeps **95%** of clean evidence-recall at $\varepsilon \approx 9.3$ and **85%** at $\varepsilon \approx 3.2$, at 2.6× chance. Utility is set by *effective contributors per bucket*: coarse buckets pool more users, so DP is nearly free; fine buckets starve, so noise bites.

What "retention" is measured against — and the cost it does* not *include. Retention throughout this paper is utility(ε)/utility(clean pool): it isolates **the cost of the privacy mechanism**, holding the architecture fixed. It is *not* the cost of the architecture, and the two should not be conflated. Bucketing into K centroids is itself a lossy retriever: on the controls, where the harness also computes an unbucketed nearest-neighbour reference over the same notes, plain kNN scores 0.43–0.65

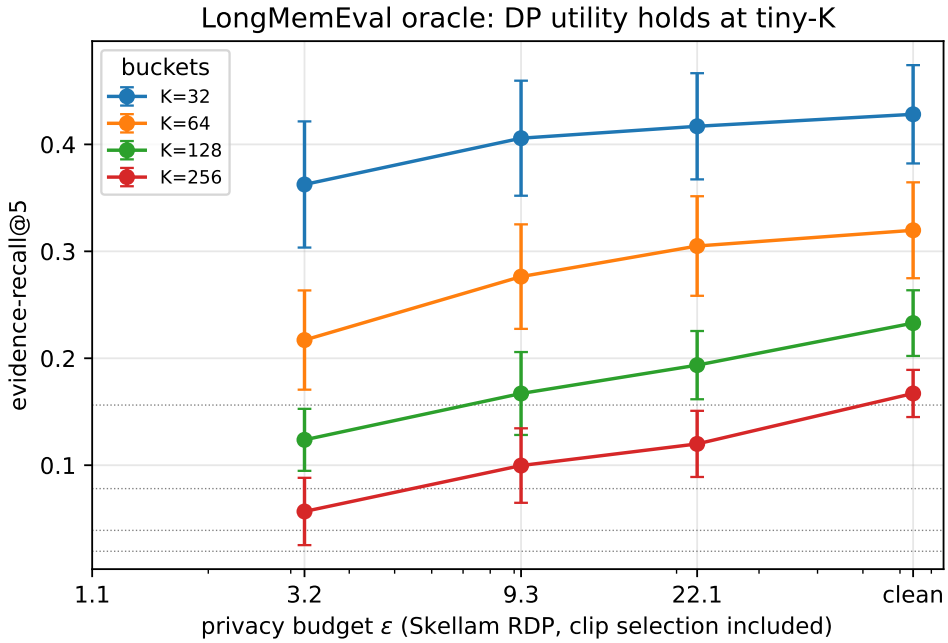

Fig. 2 Retrieval utility vs privacy budget (LongMemEval oracle, $d = 32$). At coarse $K = 32$ the private pool tracks the clean curve (95% retention at $\varepsilon \approx 9.3$) while higher-fidelity K starves buckets and DP noise bites. Dotted lines mark per-K chance. Error bars are 5-seed std. The ε axis is the rigorous Skellam-RDP budget of §5.4, clip selection included.

against the clean bucketed pool's 0.19–0.32 ($N = 100$, $K = 32$, $d = 32$–384). **Coarse bucketing costs roughly half the retrieval utility, and DP then costs a further 5%.** We state this plainly because the "95%" is otherwise easy to misread as "the private pool is 95% as good as having the notes."

That un-bucketed retriever is, however, **not an available baseline in this setting**: it requires every note in the clear at query time, which is precisely the object we may not release — it is the artifact MEXTRA consumes (§7.10). It is an upper bound, not a competitor. The honest framing is therefore: *a releasable shared pool costs about half of an unreleasable one, and making it differentially private on top of that costs 5%.* Whether the first half is worth paying is a product question about what a fleet-level routing prior is worth (§1); the second is the question this paper answers.

The d-sweep at fixed $K = 128$ shows **no dimension penalty**: retention is **72%** → **77%** → **73%** for $d = 32 \to 64 \to 128$, flat within seed noise. Under the data-dependent PCA this same sweep read 78% → 67% → 55%, which is what the superseded draft reported as a dimension effect. §7.4 explains why.

Table 8 (a) Private utility under the three projections (topic-acc @ $\varepsilon \approx 9.3$, real `all-MiniLM-L6-v2`, $N = 100$, $K = 32$; higher is better). Tiny-d wins *only* under the leaky data-dependent PCA.

d	data-PCA (leaky)	randproj (free)	publicPCA (free)
32	**0.308**	0.151	0.147
64	0.285	**0.178**	0.150
128	0.240	0.158	0.142
384 *(identity)*	0.161	0.161	**0.161**

Table 9 (b) Leakage under the three projections (tail-AUC at $K = 1024$; 0.5 = no leakage). The pre-DP gradient that motivated the crossover claim exists only under data-PCA; post-DP every cell is at chance.

	clean (pre-DP)			$\varepsilon \approx 9.3$	
d	data-PCA	randproj	publicPCA	data-PCA	randproj
32	0.944	0.988	0.975	0.522	0.499
64	0.969	0.991	0.985	0.513	0.500
128	0.985	0.992	0.991	0.540	0.545
384 *(identity)*	0.989	0.989	0.989	0.541	0.541

7.4 The dimension "crossover" is an artifact of the projection

An earlier draft of this paper reported a **dimension/density crossover**: growing d appeared to *simultaneously* raise pre-DP leakage and lower DP utility-retention, making tiny-d Pareto-preferred and, apparently, a free lunch. That finding does not survive an honest projection, and the way it fails is instructive.

The projection P was a PCA **fit on the users' own notes**. That is a data-dependent release: P is handed to every client as a public parameter while being a function of private data, so the ε of §5.4 covered only the second stage. We therefore re-ran the entire d-sweep under three projections — the original `data-PCA`, a public-seed Gaussian `randproj`, and `publicPCA` (PCA fit on a disjoint public corpus) — holding **everything else fixed**, including the DP clip bound of §5.2. At $d = 384$ the projection is the **identity**, so all three arms must coincide: that is the control, and they do (Tables 8 and 9).

Both halves of the crossover dissolve. On the **utility** axis, tiny-d wins by 91% under data-PCA (0.308 vs 0.161 at the identity), and under either data-independent projection that advantage **disappears into the seed noise**: randproj reads 0.151/0.178/0.158/0.161 and publicPCA 0.147/0.150/0.142/0.161 for $d = 32/64/128/384$, spreads of 0.03 against a 5-seed std of ± 0.02–0.03. We are deliberately careful here, because it would be easy to over-claim in the other direction: the honest statement is **not** that tiny-d becomes the *worst* choice — no ordering among d is statistically resolvable at 5 seeds under either honest projection — but that **the effect vanishes**. The crossover's utility half was manufactured entirely by the leaky projection; what replaces it is a flat line, not a reversal. On the **leakage** axis, the

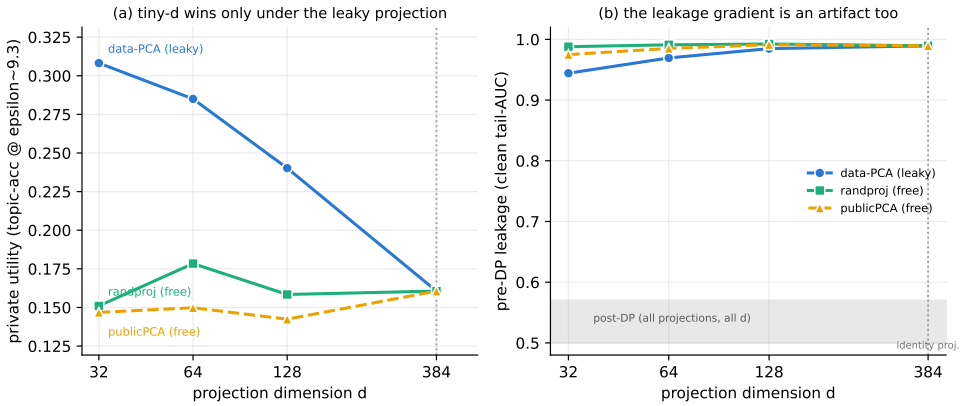

Fig. 3 The dimension crossover is a property of the projection, not of the mechanism. **(a)** Private utility: tiny-d wins only under the data-dependent PCA; under either data-independent projection it is the worst choice. **(b)** Pre-DP leakage: data-PCA shows the rising gradient the crossover claim rested on; the honest projections are saturated. All three converge at $d = 384$, where the projection *is* the identity — the control. Shaded band: post-DP tail-AUC, flat at chance for every d and every projection.

gradient that motivated the claim (0.944 → 0.989) likewise flattens to 0.988 → 0.989: once the projection stops concentrating the corpus's shared variance, small d no longer compresses away what makes a note unique. And post-DP, tail-AUC sits at **0.50–0.55 for every d and every projection**: DP pins the attack at chance regardless of dimension, so the pre-DP axis was never the operative one.

The mechanism is now visible. PCA orders directions by variance *in the users' data*; at small d it keeps exactly the directions the corpus shares and discards the idiosyncratic tail. That simultaneously flatters clean retrieval and destroys the individuating signal an attacker needs — which is precisely the "both axes at once" effect. But it purchases both with the same illicit currency: information about the private corpus, spent outside the accountant. A random projection, spending nothing, buys neither.

What survives.

Tiny-d remains a reasonable engineering default, on grounds that have nothing to do with a privacy/utility joint optimum: at $d = 32$ the payload is 12× **smaller** ($K \cdot d =$ 1,024 vs 12,288 coordinates) and post-DP leakage is **statistically indistinguishable** from the identity's (Table 9: 0.499 vs 0.541 tail-AUC at $\varepsilon \approx 9.3$, both at chance). Its private utility (0.151) is nominally 15% below the best honest cell (0.178 at $d = 64$), but as noted above that gap is inside the seed noise and we do not read an ordering from it. The case for tiny-d is therefore a **communication/compute** one — an order-of-magnitude smaller payload for no measurable privacy or utility cost — and not the free lunch the superseded draft claimed.

What this implies beyond this paper.

Data-dependent preprocessing — PCA, whitening, vocabulary selection, learned quantisers, clip bounds read off the data (§5.2) — is both an unaccounted release *and* a confound that can manufacture a dimension effect out of nothing. It is standard practice. An identity-projection control, of the kind used above, costs one extra run and would catch it.

7.5 The weak attack is blind exactly where you would deploy

The cosine-similarity score of §3 — rank a candidate by $\cos(x, \mu[\text{bucket}(x)])$ — is the measurement principle of the published agent-memory attacks [3, 4] and the natural first evaluation to reach for. It is also, we now show, **actively misleading**, and in the direction that matters: it under-reports leakage precisely in the coarse-K regime a deployment would choose. We report this as a result in its own right, because it is a trap we fell into and one the surrounding literature is arranged to fall into.

The proxy certifies a leaking pool as safe.

Compare the two attacks on the *identical clean release* (Table 4). At $K = 32$ the cosine proxy reads **AUC 0.504** — chance, to three decimals; on this evidence the pool leaks nothing and there is no privacy problem to solve. The calibrated LiRA, on that same release, reads **AUC 0.833, TPR@1%FPR 0.175 (17× the floor), and reconstructs 77% of centroids to a true in-bucket note.** The pool was never safe. The proxy simply could not see it, because an uncalibrated cosine score compares a member's similarity against a *population* baseline, and at coarse K the population baseline is high: every candidate is close to a centroid that averages hundreds of notes. Only a *per-target* likelihood ratio — calibrated against shadow releases with and without that exact note — separates "close because the bucket is broad" from "close because I am in it."

And it manufactures a false story about* where *leakage lives.

Because the proxy's sensitivity grows with K (Table 4, last column pair: 0.504 → 0.738 as K goes 32 → 2048) while the calibrated attack is already strong at $K = 32$ (0.833), a proxy-only evaluation concludes that leakage is a **sparse, high-K, low-count-tail phenomenon** — a corner case, present only in a regime whose utility retention (15–41%) nobody would ship. That is exactly the conclusion our superseded draft drew, and it is wrong twice: it understates the threat at the deployed operating point, and it invites the reader to dismiss the mechanism as solving a problem that only exists where the system is useless anyway. Table 10 preserves the proxy's numbers for the contrast.

What the proxy* does *establish, kept for the record.

On the sparse tail it is able to see ($K \geq 512$, where 2–16% of members sit in buckets with ≤ 3 contributors), clean memory re-identifies tail members at **AUC** ≈ 0.93 and DP drives that to 0.53–0.55 at $\varepsilon \approx 9.3$. We state its residual precisely rather than rounding it to "chance": the 0.03–0.05 gap above 0.5 is statistically resolvable (4.4σ

Table 10 The uncalibrated cosine proxy (LongMemEval oracle; tail-AUC over buckets with ≤ 3 contributors, 0.5 = no leakage). **Retained for contrast only.** The tail it stratifies on is *empty* at $K \leq 128$, so the proxy has nothing to report at the recommended operating point — while the calibrated attack of §7.2 breaks that same release (Table 4). Reading privacy off this table alone is the error §7.5 documents.

K	Tail %	Clean tail-AUC	$\varepsilon \approx 9.3$	$\varepsilon \approx 3.2$	Above chance
32–128	**0%**	*tail empty — proxy reports nothing here*			
512	2%	0.928 ± 0.011	0.551 ± 0.057	0.543 ± 0.045	0.9σ
1024	7%	$\mathbf{0.936 \pm 0.010}$	$\mathbf{0.530 \pm 0.014}$	0.499 ± 0.028	2.2σ
2048	16%	0.915 ± 0.010	0.541 ± 0.009	0.504 ± 0.012	4.4σ

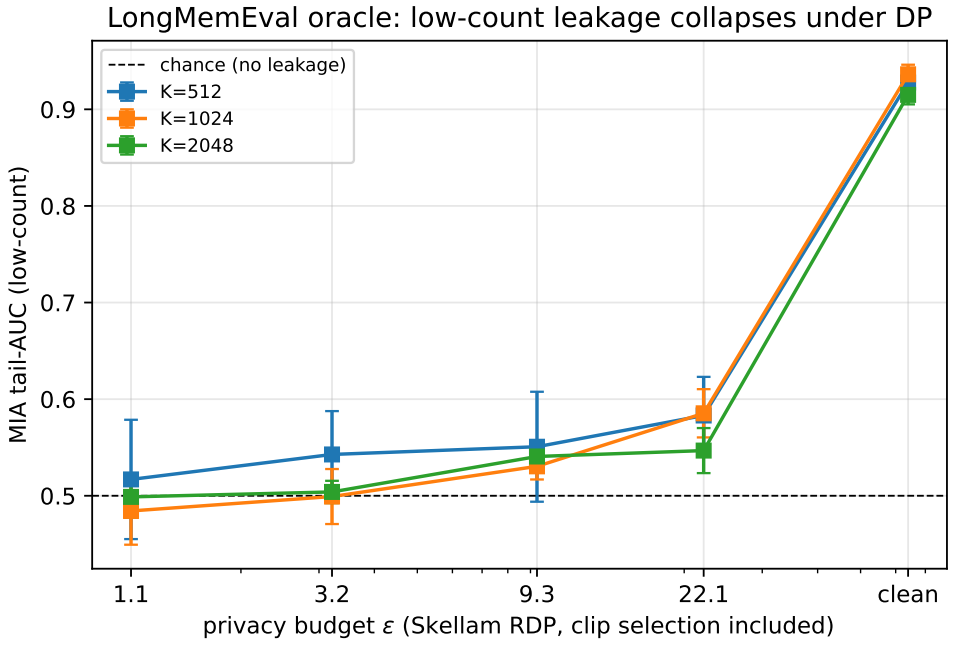

Fig. 4 The proxy's own endpoint, retained for the record: on the sparse tail it *can* see ($K \geq 512$), membership-inference AUC falls from ≈ 0.93 to 0.53–0.55 at $\varepsilon \approx 9.3$. The result is real but the lens is narrow — at $K \leq 128$ this tail is empty and the proxy reports chance, while the calibrated attack of §7.2 shows the release is leaking there too (Fig. 1b).

at $K = 2048$). The calibrated attack answers what that residual is *worth*: at the same operating point LiRA reads AUC 0.504 with TPR@1%FPR on the 1% floor. So the residual is an artifact of the proxy's weakness in *both* directions — it sees a signal that buys nothing at high K, and misses one that buys plenty at low K. This is why §7.1 headlines the calibrated low-FPR TPR throughout.

Table 11 Distilled-notes utility (answer-recall@5, vector-only release).

N (users)	K	Clean	$\varepsilon \approx 9.3$	Retention
500	32	0.451 ± 0.020	$\mathbf{0.402 \pm 0.032}$	**89%**
500	64	0.330 ± 0.025	0.266 ± 0.046	81%
100 (pilot)	32	0.496 ± 0.029	0.324 ± 0.068	65%

Table 12 Raw turns vs LLM-distilled notes (full 500-user run, $K = 1024$, tail-AUC).

Source	Clean all-AUC	Clean tail-AUC	$\varepsilon \approx 9.3$ tail-AUC	Above chance
Raw dialogue turns	0.682	0.929	0.541 ± 0.053	0.8σ
LLM-distilled notes	**0.801**	**0.963**	0.557 ± 0.026	2.2σ

This confound, like the projection, is not specific to us.

§7.4 shows a data-dependent *preprocessing* step fabricating a finding; this section shows a under-powered *attack* erasing one. Both are defaults — PCA is the obvious projection, cosine-similarity is the obvious membership score — and both bias the conclusion toward "the mechanism is fine." An evaluation that reports only an uncalibrated proxy has not measured privacy; it has measured its own attack. The remedy is cheap: our LiRA needs no model training (the aggregation is `numpy`), and 48 shadow releases run in 34 s per operating point.

7.6 Realistic payload: LLM-distilled notes

The real agent-memory payload (distilled notes) **strengthens** the case. The distillation is a fresh local-LLM run (Ollama `qwen2.5:7b`; 500 users → 6,116 notes, 100-user pilot → 1,715 notes).

Utility.

Density lifts DP retention (Table 11, Fig. 5). At full scale (500 users, denser buckets) the distilled pool retains **89% at** $\varepsilon \approx 9.3$; the 100-user pilot retains 65%; **density, not scale per se, governs retention** (more contributors per bucket ⇒ cheaper DP).

Leakage.

Distilled notes leak **more** than raw turns, yet DP kills both (Table 12, Fig. 6). Distillation **concentrates identifying facts**, so distilled memory is *more* re-identifiable pre-DP (all-AUC 0.68 → 0.80, tail 0.93 → 0.96), but the private release still drives both to within $\sim 2\sigma$ of chance. The gap is wider than the data-PCA grid showed (0.64 → 0.73), for the same reason as §7.5. The payload agents actually store is the one DP protects most decisively.

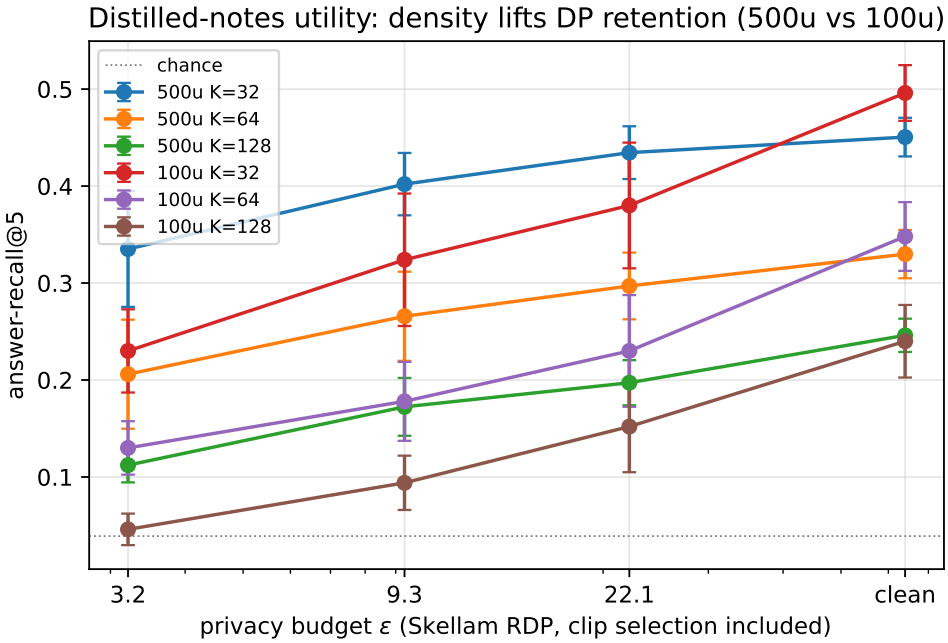

Fig. 5 Distilled-notes utility (answer-recall@5). Density governs DP retention: the dense 500-user buckets ($K = 32$) retain **89%** at $\varepsilon \approx 9.3$, whereas the sparser 100-user pilot retains only 65% at the same budget. More contributors per bucket ⇒ cheaper DP.

The calibrated attack on the realistic payload.

Table 12 is a *cosine-proxy* result, and §7.5 has just argued that a proxy-only evaluation has not measured privacy — so resting the realistic-payload claim on it alone would be exactly the error we indict. We therefore run the **calibrated LiRA** of §7.2 directly on the distilled pool, at the recommended operating point (Table 13). It confirms the claim on the instrument that counts, and it tells the same story as the raw payload does: the clean distilled pool is thoroughly broken (AUC 0.838, TPR@1%FPR 0.171 — 17× the floor — and 75% of centroids decoded to a true in-bucket note), and the private release drives it to the floor.

Two honest corrections to the proxy's story follow. First, under the calibrated attack the distilled payload is **not** meaningfully leakier than raw turns in the clear (AUC 0.838 vs 0.833; TPR 0.171 vs 0.175) — the proxy's "distillation concentrates identifying facts" gradient (0.68 → 0.80 all-AUC) does not survive calibration, and we no longer claim it. Second, the distilled payload is if anything *better* protected on the extraction axis (0.302 vs 0.396 decode at $\varepsilon \approx 9.3$), plausibly because distilled notes are semantically denser and less redundant, so a noised centroid is a worse pointer to any one of them. What survives, and is the claim worth making, is the one that matters for deployment: **on the payload agents actually store, the calibrated attack is defeated at the same operating point and the same budget as on raw turns.**

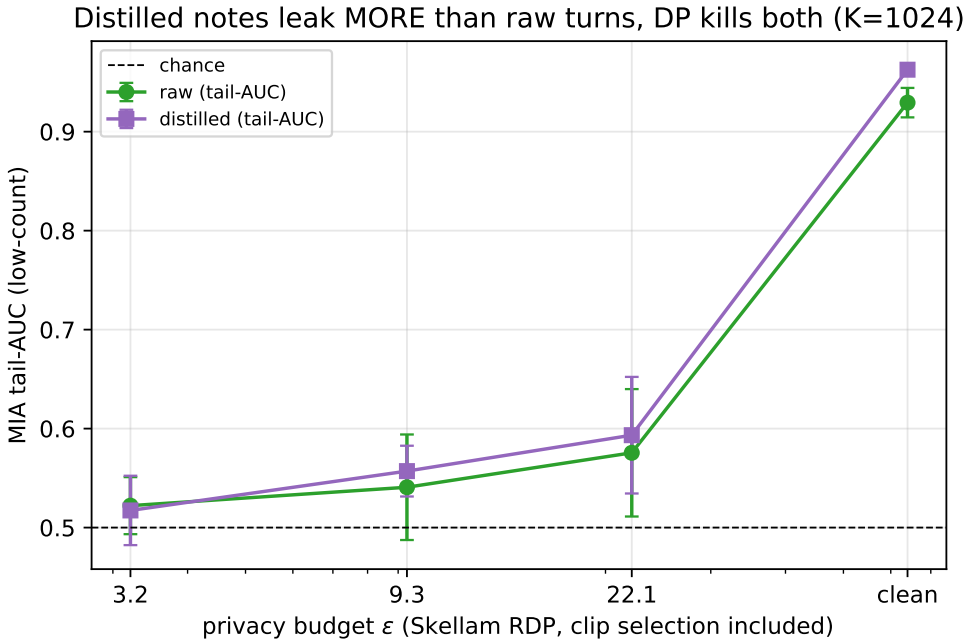

Fig. 6 Distilled notes leak *more* than raw turns in the clear (distillation concentrates identifying facts: clean tail-AUC 0.93 → 0.96), yet the SecAgg+Skellam release collapses *both* to near-chance at $\varepsilon \approx 9.3$. The realistic payload strengthens, not weakens, the result.

Table 13 Calibrated LiRA on the LLM-distilled payload (500 users, 6,116 notes, $K = 32$, $d = 32$; 3 seeds × 48 shadows) — the realistic agent-memory object, measured with the attack §7.5 shows to be the only trustworthy one. The raw-turn row is Table 4's $K = 32$ row, repeated for comparison at the *same* operating point.

Payload	Release	AUC	TPR@1%FPR	Decode-extract
Raw dialogue turns	clean	0.833	0.175	0.771
Raw dialogue turns	$\varepsilon \approx 9.3$	0.502	**0.011**	0.396
LLM-distilled notes	clean	**0.838**	**0.171**	**0.750**
LLM-distilled notes	$\varepsilon \approx 9.3$	0.503	**0.011**	**0.302**

7.7 Accounting results

The total ε decomposes cleanly, and three of its four terms are zero or negligible (Table 14).

- **Discretisation is free.** At `range_max` $= 10^6$ the discrete Skellam ε equals the Gaussian-RDP ε to nine decimals; the integer/SecAgg quantisation costs no privacy.

Table 14 ε decomposition at $\sigma = 0.606$, $\delta = 10^{-5}$, $K = 32$, $d = 32$. The reported budget covers the *whole* pipeline, not just its last stage.

Term	ε	Note
Continuous-Gaussian RDP reference	9.2909	—
+ discretisation (Skellam)	9.2909	surcharge 1.3×10^{-9}: free at `range_max` $= 10^6$
+ projection (public seed)	9.2909	data-independent ⇒ +0.000
+ DP clip-bound selection ($\varepsilon_c = 0.1$)	**9.3109**	+0.020 **(0.2%)**

- **The projection is free.** The public-seed random projection is data-independent, so it adds zero ε — unlike the PCA it replaces, whose cost was unbounded and unaccounted (§7.4).
- **The clip bound is nearly free in ε, and cheap in utility.** DP-selecting C with the exponential mechanism at $\varepsilon_c = 0.1$ costs **0.2%** of the budget (§5.2), and $B := C$ costs nothing at all. The selection is conservative — it over-estimates C by $\sim$1.6× and therefore over-noises — which is what turns 0.416 into **0.406** at $K = 32$ and 0.117 into 0.100 at $K = 256$. The superseded draft read C and B straight off the private data and charged neither.
- **Vector-only dominates, at a fixed occupancy rule.** At matched σ the vector-only release gives **identical recall to the last decimal** as the two-channel design, at ε 14.0 → 9.3, because the count *magnitude* only ever cancelled under normalisation. One composition, $\sim$1.5× tighter ε, same utility. The one thing the count channel did supply is **occupancy** — which buckets are non-empty — and that does not cancel. At the K we recommend and report, no bucket is empty, so the rule is vacuous and the domination is unconditional; at high K an occupancy channel must be bought, and §7.11 measures what it costs and how well it works.
- **Repeated release is the one term that is *not* cheap — and the one a memory pool cannot avoid.** Every entry above prices a *single* release. A deployed memory is re-aggregated as users write notes, and RDP composes additively, so T naive re-releases at $\sigma = 0.606$ cost $\varepsilon \approx 22/44/93/153$ for $T = 4/12/30/52$ (Table 3). This dwarfs every other term in this table by two orders of magnitude, and it is the term most easily left implicit. It is, however, *payable*: the required σ grows only as $\sqrt{T}$, so a cadence can be bought up front — **monthly re-aggregation for a year at 81% retention, inside the same $\varepsilon \approx 9.31$** (§5.7). The public parameters do not recompose (C is selected once; the projection and anchors are public-seed), so the ε_c above is charged once regardless of T.

7.8 Robustness

Every number above is a **5-seed mean with the final (not peak) metric**, reported with its std. The refactor that introduced the `--proj` flag was validated by re-running `--proj pca` and checking it reproduced the previously released grid **bit-identically** (verified on the utility, leakage and distilled paths, to 10^{-9}); the projection is therefore the only variable in the §7.4 ablation. The subsequent switch to a DP-selected clip

Table 15 At-scale utility (LongMemEval **s**, 246k notes; evidence-recall@5, $d = 32$, 5-seed).

K	Chance	Clean	$\varepsilon \approx 9.3$	$\varepsilon \approx 3.2$	Retention@$\varepsilon \approx 9.3$
32	0.156	0.422 ± 0.056	0.421 ± 0.059	0.422 ± 0.058	**100%**
64	0.078	0.309 ± 0.041	0.312 ± 0.041	0.301 ± 0.037	**101%**
128	0.039	0.223 ± 0.023	0.218 ± 0.023	0.201 ± 0.027	**98%**
256	0.020	0.159 ± 0.020	0.154 ± 0.018	0.134 ± 0.022	**97%**

Table 16 At-scale leakage (LongMemEval **s**). The vulnerable tail is empty, and membership inference is at chance even on the clean pool.

K	Low-count tail %	Clean all-AUC	$\varepsilon \approx 9.3$ all-AUC
512	0.0%	0.508	0.508
1024	0.0%	0.519	0.505
2048	0.0%	0.529	0.502

bound (§5.2) changes every arm identically. Re-running the full grid at 5 seeds reproduced every qualitative headline with **no sign flip** except the dimension crossover of §7.4, which reversed — the point of that section.

7.9 At-scale generality (LongMemEval s, 246k notes)

We repeat the utility and leakage measurements on the **full s variant** — the same 500 users but **246,073 notes** (∼96% distractors), ∼22× the oracle corpus. Both axes behave exactly as the density argument of §8 predicts.

Utility is free.

See Table 15. The 100–101% cells are *not* evidence that noise helps: the clean/private differences (≤ 0.003) are an order of magnitude below the 5-seed std (0.02–0.06). The honest reading is that at this density **DP costs nothing measurable**.

The vulnerable tail vanishes.

At this density essentially every bucket has many contributors, so the low-count tail (≤ 3 contributors) is **empty (0% of members)** across $K \in \{512, 1024, 2048\}$, and membership inference is **already at chance without DP** (Table 16).

How to read this — corrected.

The superseded draft used this run to argue that the leakage-drop is "really" a sparse-tail phenomenon, and that the dense regime is therefore *safe on both axes for free* because averaging over hundreds of contributors already destroys the membership signal. **That reading was an artifact of the proxy** (§7.5), and we withdraw it. What Table 16 shows is that the *proxy's tail statistic* vanishes at density — not

Table 17 The published attacks against a live agent (60 users, 883 distilled notes, $K = 256$, $d = 32$, the paper's mechanism — randproj + DP clip; 40 extraction and 120 membership trials per seed, 3 seeds).

Shared-memory back-end	MEXTRA recovery	MRMMIA-AUC
Raw text (baseline, no privacy)	$\mathbf{1.000 \pm 0.000}$	$\mathbf{0.981 \pm 0.002}$
Our SecAgg+Skellam pool ($\varepsilon \approx 9.3$)	$\mathbf{0.017 \pm 0.012}$	0.526 ± 0.044

that the pool stops leaking. The calibrated attack breaks dense, coarse-K releases: at $K = 32$ on the oracle corpus, where each bucket averages ∼340 contributors and the cosine proxy reads 0.504, LiRA reads AUC **0.833** with a 17×-above-floor TPR (§7.1). Averaging does not anonymise; it only defeats an adversary who scores against a population baseline.

The at-scale run therefore **confirms the utility half and bounds the proxy**, rather than bounding the result: at this density DP costs *nothing measurable* on retrieval (97–100% retention, Table 15), which is the finding worth carrying. The leakage half of the table should be read as a statement about the instrument, not about the release — and it is one more reason we headline the calibrated attack throughout.

7.10 The published attacks against a live agent (MEXTRA / MRMMIA)

The attacks so far operate on the released embeddings. To close the loop on the *published* threat model — a live LLM agent that stores and serves memory as **text** — we build an A-MEM/Mem0-style shared-memory agent (local `qwen2.5:7b`) and run both attacks end-to-end. The agent retrieves the top-k shared-memory notes for a query into its context and answers; the adversary issues (a) a **MEXTRA** [3] extraction prompt that asks the agent to reproduce its memory verbatim, and (b) a **MRMMIA** [4] membership probe for a candidate note. We hold the distilled notes fixed and swap only the shared-memory back-end (Table 17).

On the raw-text agent both published attacks succeed decisively: every targeted member note is reproduced verbatim and membership is inferred at AUC 0.98. Against our release both collapse: membership is at chance (0.526 ± 0.044, straddling 0.5 across seeds: 0.528/0.471/0.578).

The private-column MEXTRA figure is **0.017, not 0**, and the distinction matters. The centroid pool contains no member text, so verbatim recovery is *structurally* impossible: the agent's context holds only public notes decoded from centroids. The 1.7% is therefore the **false-positive rate of the recovery detector** — a decoded public note that happens to sit within the 0.9 embedding-similarity threshold of a target member note. It measures our measurement, not the release. We report it rather than round it to zero.

A harness bug we found and fixed. The membership probe originally drew its non-members from the same held-out set that served as the adversary's public decode corpus. In `private` mode the agent's context *is* that corpus, so non-member candidates appeared verbatim in the context while members never could, and the attack scored **below** chance

(MRMMIA-AUC 0.31–0.35 — an inverting adversary would have read 0.65–0.69). Any AUC that lands reliably below 0.5 is a design error, not a privacy result. The corpus is now split three ways — members / non-members / public decode targets, pairwise disjoint — which is what the table above reports. The superseded draft's 0.454 carried the same confound.

(This is a demonstration at modest scale, not a tuned attack sweep; a stronger prompt-injection adversary against the raw baseline would only widen the gap, since our release exposes no text either way.)

How to read this.

MEXTRA is a prompt-injection attack that induces an agent to echo its **raw-text** memory. Our release stores no text, so the correct claim is not that we *defeat* MEXTRA but that we **structurally immunise** the agent against it: the attack's target object no longer exists. Stating it otherwise would be a strawman, since any vector-only datastore inherits the same immunity for free. The adversary that a vector payload genuinely must answer to is **embedding inversion** — recovering a note from the released centroid. That is the reconstruction-decode attack of §7.2, and we run it in its *strongest closed-set form*: the adversary is handed the true candidate note set and need only pick the right element, which upper-bounds free-form inversion (e.g. `vec2text`-style decoders) on the same release. It recovers **91%** of clean centroids and **0.003** under ours. §7.2, not this section, is therefore the extraction result that carries weight for our mechanism; this section's role is to show what the *status-quo* agent leaks, and that the artifact those published attacks consume is simply gone.

7.11 Occupancy: what the count channel actually buys

An empty bucket is the one place the count channel is not redundant: `normalize` turns a pure-noise $S[k]$ into a unit vector that competes for top-k slots against genuine centroids. We measure (a) how often empty buckets occur, and (b) whether they can be detected *privately*. Real LongMemEval-oracle (10,957 notes, 500 users, $d = 32$), the paper's mechanism (randproj, DP clip), $\sigma_v = 0.606$ ($\varepsilon \approx 9.3$); occupancy over 5 anchor seeds, detection on a single release. Reproduce with `scripts/agentmem/_occupancy_channel.py`.

(a) At the operating points, there are none.

See Table 18. Every retrieval number in this paper is reported at $K \leq 256$, where at most **one bucket of 256** is ever empty; at the recommended $K = 32$ there are none on any seed. The noisy-normalisation failure mode is **absent from the reported utility** rather than suppressed by tuning.

(b) Where it does occur, occupancy is expensive to release.

See Table 19. Three readings, all load-bearing. First, the right channel is the **binary user-indicator** (each user contributes 0/1 per bucket), not the raw count: it roughly halves the sensitivity and buys 5–9 AUC points at identical ε. Second, it is still **not free**, and the intuition that it should be is a note-level intuition. Under **user-level** DP,

Table 18 Empty-bucket rate by fidelity K (5 anchor seeds). Every retrieval number in this paper is reported at $K \leq 256$, where at most one bucket of 256 is ever empty.

K	32	64	128	256	512	1024	2048
Empty buckets (%)	**0.00**	**0.00**	**0.00**	0.23 ± 0.19	2.66 ± 0.88	10.14 ± 1.53	22.91 ± 2.01

Table 19 Occupancy-detection AUC (empty vs occupied; 0.5 = undetectable), against the total ε of the release. No rule is simultaneously free and accurate.

Occupancy rule	Sens. C_c	ε	$K = 512$	$K = 1024$	$K = 2048$
$\|\|S[k]\|\|$ vs floor $\sigma C\sqrt{d}$ (free: post-proc.)	—	**9.31**	0.497	0.485	0.481
Raw-count, $\sigma_c = 2$ (cheap 2nd comp.)	20.2	9.78	0.695	0.561	0.550
User-indicator, $\sigma_c = 2$	9.7–12.4	9.78	0.723	0.593	0.549
Raw-count, $\sigma_c = \sigma_v$ (full 2nd comp.)	20.2	13.94	0.824	0.683	0.633
User-indicator, $\sigma_c = \sigma_v$	9.7–12.4	13.94	**0.873**	**0.764**	**0.698**

deleting a user erases every entry of its indicator vector at once, so the L2 sensitivity is $\sqrt{b_u}$ — about 5–8 here, and its DP-selected public bound lands at 9.7–12.4, not 1 — and a full second composition (ε 9.31 → 13.94) is what buys AUC 0.698 where it is most needed ($K = 2048$, 23% empty). Third, the free rule — thresholding the released vector's own norm against $\sigma C\sqrt{d}$, which costs no privacy because S is already public — is **at chance** (0.48–0.50) under the final mechanism: the DP-selected clip bound is deliberately conservative, and the extra noise it buys erases the norm signal entirely.

Neither is an implementation shortfall. The DP noise that drives membership inference to chance in a one-contributor bucket (§7.5) is *the same noise* that hides whether the bucket has a contributor at all: **occupancy and membership are the same signal**, and a mechanism cannot hide the member while revealing the bucket. The design consequence is to operate at dense K, where retention is highest anyway (§7.3), rather than to buy a count channel that cannot pay for itself.

8 Discussion and Limitations

The endpoint is* not *confined to the sparse tail — and believing it was is what a weak attack does to you.

An earlier version of this paper conceded here that the 0.9 → 0.5 drop "lives in the low-count tail," that dense buckets are protected by averaging anyway, and that the result should therefore be framed narrowly as *"DP protects the vulnerable tail a sum mechanism most exposes."* **We withdraw that concession.** It was an artifact of measuring with an uncalibrated cosine proxy, which reports chance (0.504) at $K = 32$ on a release the calibrated attack breaks at AUC 0.833 with 17×-above-floor TPR and 77% note reconstruction (§7.1, §7.5). Averaging over a coarse bucket does *not* protect its members; it only defeats an attacker who compares against a population baseline instead of a per-target one. The honest frame is the stronger one: **the clean pool**

leaks at every fidelity we measured, and DP pins the calibrated attack at the false-positive floor at every fidelity we measured — the privacy axis is flat in K, and the operating point is chosen on utility alone. The at-scale **s**-variant (§7.9) remains a genuine boundary on the *proxy*'s tail statistic (with 246k notes the low-count tail is empty), but it is not, as we previously wrote, a boundary on the result.

Attack strength.

We evaluate at three escalating strengths and the endpoint holds at each: (i) an embedding-similarity proxy (§7.4, §7.5), which we now report chiefly as a *cautionary* instrument rather than as evidence; (ii) a **calibrated offline-LiRA** MIA reported at low-FPR TPR plus a reconstruction-decode extraction attack (§7.2), which reveal the clean pool is *far* more leaky than the proxy showed — at the recommended $K = 32$, TPR@1%FPR 0.175 and 77% reconstruction where the proxy saw nothing at all — yet still fall to the false-positive floor under DP; and (iii) the **published MEXTRA/MRMMIA attacks against a live agent** (§7.10), which fully extract a raw-text memory (recovery 1.0, AUC 0.98) but are driven to their detector's noise floor against the centroid release (recovery 0.017, AUC 0.526). We note that this demonstration runs at $K = 256$ with 60 users and 883 distilled notes — a smaller and sparser configuration than the $K = 32$ operating point of §7.1, and one for which we report no utility — so it should be read as evidence about the *status-quo raw-text* agent, not as a second measurement of our recommended release. The endpoint thus **strengthens** as the attack gets stronger, which is the signature one wants: a defence that only survives weak adversaries usually fails on the first strong one, and ours does the opposite. Limits of (iii): it is a modest-scale demonstration on one open model with untuned extraction prompts; a stronger prompt-injection adversary would widen, not close, the gap, since our release exposes no text regardless. A user-level (rather than note-level) live-agent attack is left to future work.

Reconstruction and membership do not die together at coarse K.

Stated plainly because it is the one place the recommended point is not fully clean: at $K = 32$, DP drives *membership* to the floor (TPR@1%FPR 0.175 $\rightarrow$ 0.011) but only halves *reconstruction-decode* (0.771 $\rightarrow$ 0.396, against a $\approx$0.08 noise floor). The residual is largely a property of the metric at coarse K — a centroid there averages $\sim$340 notes, so "the top-1 decode is a true in-bucket member" is partly satisfiable from bucket breadth alone, and the disclosure is region membership rather than user identification — but it is above floor, and we do not round it away. A deployment for which reconstruction is the operative threat should run at $K = 128$, where decode reaches its floor (0.070) and membership stays at it, for 72% retention instead of 95% (§7.1).

Scope: the pool is a routing prior, not a note exchange.

The release is K centroid vectors and **no text**, so a receiving agent cannot read another user's note out of it, and we make no such claim (§1). Utility is accordingly measured as *routing* fidelity — did the query reach the bucket holding its evidence — not end-to-end downstream QA; extending to generated-answer quality is future work,

as is a text-bearing second tier, which would need its own budget (§8, "Stationarity and fidelity"). Retrieval is bucket-granular, not exact-note ranking. This is a narrower product than "every agent inherits every fix," and it is the one the mechanism can carry under a guarantee.

Deployment gaps we specify but do not measure.

Four, stated plainly. (i) **Client dropout:** our runs assume full participation. A naive μ/N noise share loses the guarantee precisely when turnout falls — at 50% survival the true budget is $\varepsilon \approx 13.9$, not the nominal 9.3 — so a deployment must use the $M_{\min}$-calibrated share or fault-tolerant noise reconstruction of §5.6. We specify both; we measure neither. (Repeated release, the sibling of this gap, we now *do* account for and measure: §5.7.) (ii) **Occupancy at high K:** the leakage tables release the vector-only pool with *no* occupancy suppression, so they contain no oracle; dropping the gate an earlier draft used moves every tail-AUC by ≤ 0.003 (§5.3). What a dense high-K deployment needs for *retrieval* — a private occupancy gate to keep noise-only buckets out of top-k — is a real cost we do not pay here (we report retrieval only at $K \leq 256$, where no bucket is empty), and §7.11 prices it: no rule is simultaneously free and accurate, because occupancy and membership are the same signal. The honest reading is that near-empty buckets cannot be *both* privately aggregated and privately enumerated. Note this cost is now easier to avoid than to pay: §7.1 shows the frontier's optimum is at coarse K anyway. (iii) δ **at production scale:** $\delta = 10^{-5}$ is 200× below $1/N$ at our $N = 500$ **users**, but a 10^5-user fleet must retighten it (§5.4), at a cost of $< 23\%$ ε. (iv) **Drift across a multi-round deployment:** §5.7 prices the *budget* of T releases but holds the corpus fixed; it does not model a fleet whose topics move between rounds, which would additionally require the periodic re-seeding of the (public, zero-cost) bucket geometry discussed below. The composition result is therefore sound as an accounting statement and untested as a longitudinal one.

On the choice of $\varepsilon \approx 9.3$.

It is not a tight budget by the standards of the DP literature, and we do not present it as one. Two things should be weighed against it. First, it is an **end-to-end** budget: it covers the projection, the LSH anchors, the clip bound and the quantiser bound, every one of which is public or DP-selected (§7.7). The smaller ε values commonly reported for pipelines of this shape typically account only for the final noise addition and silently exclude a data-dependent preprocessing step of exactly the kind §7.4 shows to be both leaky *and* distorting — so the numbers are not comparable in the direction a reader would assume. Second, the empirical endpoint is what the budget is *for*: at this ε, every attack we can mount — including the calibrated LiRA, the reconstruction decode, and both published attacks against a live agent — is driven to its floor. We report the budget honestly and let the measured attack surface, rather than the number alone, carry the claim.

The cost of honesty, and what it buys.

Closing the two unaccounted releases — the data-fit projection (§4) and the data-read clip bound (§5.2) — lowers every clean-utility number in §7 by roughly a quarter

($K = 32$ evidence-recall $0.577 \to 0.428$) and is the right trade. The ε of §5.4 now covers the whole pipeline; the clip selection costs 0.2% of it and $B := C$ costs nothing; the leakage-drop endpoint (§7.5) and the calibrated attack (§7.2) both get *stronger*, because the discarded PCA had been pre-anonymising the corpus for free. We report the superseded data-PCA grid alongside, as the §7.4 ablation, rather than quietly deleting it. The one casualty is the dimension crossover, an artifact of the discarded step; we treat its refutation as a result (§7.4) rather than a footnote.

We have not tried **DP-PCA** — fitting the projection under its own privacy budget — which is the only route we know of that would recover the variance concentration soundly, at the cost of a larger ε and a second mechanism to account for. That is the natural next step, and §7.4's ablation quantifies exactly how much utility is on the table ($0.151 \to 0.308$ in private topic-accuracy at $d = 32$, if it could be had for free — which it cannot).

Stationarity and fidelity.

Two structural assumptions. The bucket geometry is fixed for an aggregation epoch, so a fleet-wide topic shift can starve some buckets (noise dominates) and crowd others (the global clip bites). Because both the anchors and — once the fix above is applied — the projection derive from a public seed or public data, the geometry can be **re-initialised periodically at zero privacy cost**, which is the natural remedy; we do not evaluate drift. Separately, $K = 32$ is a coarse memory: retrieval is bucket-granular and the pool is 32 vectors. Finer semantic resolution need not come from pushing K into the regime where buckets starve (§7.3); a two-tier pool — coarse, dense, high-retention buckets for routing, then a sub-pool consulted after routing — is the natural architecture, but the second tier needs its own privacy budget and is not free. We leave it to future work.

Non-novel components, explicitly.

The dimension dependence of DP is classical [16, 17]. We had claimed a *coupled* manifestation across leakage and utility in agent memory; §7.4 retracts that claim and shows the coupling was induced by a data-dependent projection. What we now claim on this axis is the refutation and the control that exposes it. The SecAgg + Skellam path is prior work [8, 9, 14], and the LSH-bucketing-plus-DP skeleton overlaps the centralized *DP Datastore Generation* [7] (§2); the novelty is the federated/curator-free SecAgg+Skellam composition, the agent-memory payload, the joint frontier and its measured attack-drop endpoint, and the two methodological refutations (§7.4, §7.5).

What we would most like to be checked.

Two claims carry the paper and both are cheap to falsify, which we regard as a feature. (i) That the **cosine proxy is blind at coarse** K (§7.5) — a single LiRA run at the reader's own operating point settles it, in under a minute on CPU. (ii) That the **data-dependent projection manufactures the dimension effect** (§7.4) — an identity-projection control at $d = D$, which all projections must agree on, settles that. Both are one extra run. Both, in our case, overturned a result we had already written up.

9 Conclusion

A shared LLM-agent memory pool can be made **useful and provably private at the same operating point** — a claim that only means anything if both axes are measured there, which is the discipline this paper is organised around. Aggregating per-user bucketed memory embeddings under Secure Aggregation and the Skellam mechanism yields a curator-free central-DP **routing prior**: K centroid vectors, no text, queryable by any agent in the fleet. On the joint frontier (§7.1) the coarse-K regime is usable and safe together — at $K = 32$ the pool retains **95% of clean evidence-recall at** $\varepsilon \approx 9.3$ while a calibrated likelihood-ratio attack falls from **AUC 0.833 and a** $17\times$**-above-floor true-positive rate to the 1% false-positive floor**, and at $K = 128$ (72% retention) reconstruction reaches its floor as well. The privacy axis is **flat in** K: DP defeats the calibrated attack at every fidelity we measured, so K is chosen on utility alone. Every parameter of the release is public or DP-accounted, so that ε is the whole cost — the discrete mechanism is privacy-free at our quantiser resolution, the public-seed projection is privacy-free by construction, the clip bound costs 0.2% of the budget, and a vector-only release gives identical retrieval at $\sim 1.5\times$ tighter ε. Because a memory is re-aggregated rather than released once, we compose over the deployment: a naive cadence destroys the guarantee (ε 9.3 $\rightarrow$ 93 for a month of daily rounds), but the cost is only $\sqrt{T}$ in the noise scale, so a **monthly re-aggregation sustained for a year fits inside the same** $\varepsilon \approx 9.3$ **at 81% retention** (§5.7). The endpoint holds on the payload a production agent actually stores: run directly on LLM-distilled notes, the calibrated attack is defeated at the same operating point and the same budget (§7.6).

We also report **two negative results we think travel**, both of which overturned findings we had already written up, and both of which a single control run would have caught. The dimension/density "crossover" that made tiny-d look like a joint privacy/utility optimum is an artifact of fitting the projection on user data — an unaccounted release that flatters the clean baseline and pre-anonymises the corpus at once; an identity-projection control catches it (§7.4). And the uncalibrated cosine proxy that agent-memory work reaches for by default is **blind at exactly the operating point one would deploy**: it reports chance (0.504) at $K = 32$ on a release the calibrated attack breaks at AUC 0.833, and it thereby manufactures the comfortable conclusion that leakage is a sparse-regime curiosity rather than a property of the pool you would actually ship (§7.5). The common shape is worth naming: *a default choice — the obvious projection, the obvious attack — that biases the evaluation toward "the mechanism is fine."* Across three escalating attack strengths the endpoint instead *strengthens*: the raw-text memory of a live agent is fully extractable (recovery 1.0, AUC 0.98), while our release, which contains no note text at all, drives both published attacks to their detector's noise floor.

Appendix A Reproducibility

Active code lives in `scripts/agentmem/` (see `scripts/README.md`); the pooled crypto is `qpriviot_fl/privacy_utils.py`.

- **Utility**: `_longmemeval_probe.py` (real), `_agentmem_probe.py` (controls), `_longmemeval_distilled_utility.py` (distilled).
- **Leakage (proxy)**: `_longmemeval_leakage.py`, `_agentmem_leakage.py`, `_longmemeval_distilled_analysis.py` (raw-vs-distilled).
- **Leakage (calibrated, §7.2)**: `_agentmem_lira.py` — offline-LiRA MIA (AUC + low-FPR TPR) and reconstruction-decode extraction, via shadow releases.
- **Leakage (live agent, §7.10)**: `_agentmem_llm_attack.py`, an A-MEM/Mem0-style shared-memory agent (local Ollama) under end-to-end MEXTRA extraction + MRMMIA membership prompts, contrasting a raw-text memory vs our centroid release. Takes `--proj`/`--clip-eps` like the rest; the note corpus is split three ways (members / non-members / public decode corpus, pairwise disjoint), without which the membership probe scores below chance (§7.10). Results in `experiment_results/llm_attack_rp/`.
- **Joint frontier (§7.1) and multi-round (§5.7)**: `scripts/shell/rerun_joint.sh` fills the cells the headline grid omits — utility at $K \geq 512$, the cosine proxy at $K \leq 256$, and the *calibrated* LiRA at $K \leq 256$ — into `experiment_results/joint_rp/`, and prices the T-round schedule by injecting the solved σ via `--fl_sigma`. `_joint_frontier.py` regenerates Table 4, Table 3 and `figures/fig_joint_frontier.pdf` from those JSONs (`--latex` emits the table bodies). The LiRA-at-low-K runs are the load-bearing ones: they are what shows the cosine proxy of `_longmemeval_leakage.py` to be at chance on a release the calibrated attack breaks (§7.5).
- **Accounting**: `_skellam_accounting.py` (Skellam-RDP ε; discretisation-free check). Multi-round composition is the `releases=` argument of `skellam_rdp_epsilon(...)`, already present in `privacy_utils.py`; the σ column of Table 3 is a bisection against it (`sigma_for()` in `_joint_frontier.py`).
- **Distillation**: `_longmemeval_distill.py` turns LongMemEval turns into A-MEM/Mem0-style notes via a local Ollama model (`qwen2.5:7b`); it checkpoints incrementally.
- **Projection (§4, §7.4)**: every script takes `--proj {randproj,pca,publicpca}`; `randproj` is the public-seed, data-independent default (`make_projector` in `_agentmem_probe.py`, `PUBLIC_PROJ_SEED`). `--proj pca` reproduces the superseded data-dependent grid **bit-identically**, which is how the §7.4 ablation isolates the projection as the only variable.
- **Clip bound (§5.2)**: every script takes `--clip-eps` (default `0.1`). `dp_quantile_clip` in `qpriviot_fl/privacy_utils.py` is the exponential mechanism over the public grid `PUBLIC_CLIP_GRID`; `skellam_rdp_epsilon(..., clip_eps=, clip_releases=)` folds its cost into the reported ε. `--clip-eps 0` reproduces the superseded, unaccounted empirical p95.
- **Drivers → tables/figures**: `scripts/shell/rerun_final.sh` runs the headline 5-seed grid into `experiment_results/rerun_grid_rp/*.json`, the at-scale `s`-variant into `rerun_grid_s_rp/*.json`, and the two §7.4 ablation arms into `rerun_grid_abl_pca/` and `rerun_grid_abl_pp/` (identical to the headline except for `--proj`). `scripts/shell/rerun_lira.sh` (with `--proj randproj --clip-eps 0.1`) runs the calibrated-attack grid (§7.2)

into `experiment_results/lira_rp/*.json`. `rerun_figures.py --grid rerun_grid_rp` regenerates `figures/fig_*`. The superseded grids (`rerun_grid`, `rerun_grid_s`, `lira`) are retained only to certify the bit-identical `--proj pca` check of §7.8. Each script has an additive `--json` dump; the crypto path is byte-identical to the released `privacy_utils`.

- **Occupancy channel (§7.11)**: `_occupancy_channel.py`.
- **Occupancy gate (§5.5)**: `_longmemeval_probe.py` takes `--gate {none,oracle}`, default `none`. `none` releases *every* bucket, so an empty one carries normalized Skellam noise; this is the mechanism of Algorithm 1 and it is what every retrieval number in this paper is measured under. `oracle` suppresses empty buckets using the clean counts — a non-private read of the corpus, and therefore an unaccounted release. It is vacuous at $K \leq 256$ (identical numbers, since no bucket is empty) and reproduces the superseded high-K cells.

Run: `bash scripts/shell/rerun_final.sh && bash scripts/shell/rerun_joint.sh && python scripts/agentmem/rerun_figures.py --grid rerun_grid_rp` (and `bash scripts/shell/rerun_lira.sh` for §7.2; §7.10 needs a local `ollama serve` with `qwen2.5:7b`).

Declarations

- **Funding.** No funding was received for conducting this study.
- **Competing interests.** The author declares no competing interests.
- **Ethics approval and consent to participate.** Not applicable. This study involves no human participants and no animals. All corpora are pre-existing, publicly released datasets; no new data were collected from individuals.
- **Consent for publication.** Not applicable.
- **Data availability.** All datasets used are public: LongMemEval (oracle and `s` variants) [21] and 20-Newsgroups. The LLM-distilled note corpus is regenerated by `scripts/agentmem/_longmemeval_distill.py`.
- **Materials availability.** Not applicable.
- **Code availability.** The full harness, experiment grid and figure/table generators are released with the paper (§A).
- **Author contributions.** N.S. designed the mechanism, implemented the harness, ran the experiments and wrote the manuscript.

References

[1] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., Zhang, Y.: A-MEM: Agentic memory for LLM agents. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 38 (2025) arXiv:2502.12110

[2] Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D.: Mem0: Building production-ready AI agents with scalable long-term memory. In: Proceedings of the 28th European Conference on Artificial Intelligence (ECAI). Frontiers

in Artificial Intelligence and Applications, vol. 413 (2025) arXiv:2504.19413. https://doi.org/10.3233/FAIA251160

[3] Wang, B., He, W., Zeng, S., Xiang, Z., Xing, Y., Tang, J., He, P.: Unveiling privacy risks in LLM agent memory. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL) (2025) arXiv:2502.13172. https://doi.org/10.18653/v1/2025.acl-long.1227 . Introduces the MEXTRA memory-extraction attack

[4] Chen, K., Pang, Y., Wang, T.: MRMMIA: Membership inference attacks on memory in chat agents. arXiv preprint (2026) arXiv:2605.27825

[5] Rezazadeh, A., Li, Z., Lou, A., Zhao, Y., Wei, W., Bao, Y.: Collaborative memory: Multi-user memory sharing in LLM agents with dynamic access control. arXiv preprint (2025) arXiv:2505.18279

[6] Chen, Y., Zhao, J., Tang, B., Wang, H., Zhang, Y., Huang, F., Xiong, F., Li, Z.: MemPrivacy: Privacy-preserving personalized memory management for edge-cloud agents. arXiv preprint (2026) arXiv:2605.09530

[7] Abouelenein, A., Torki, M.: Differentially private datastore generation for retrieval-augmented inference. arXiv preprint (2026) arXiv:2606.01413. Accepted at ICPR 2026

[8] Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1175–1191 (2017). https://doi.org/10.1145/3133956.3133982

[9] Agarwal, N., Kairouz, P., Liu, Z.: The skellam mechanism for differentially private federated learning. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 34 (2021) arXiv:2110.04995

[10] Hou, C., Wang, M.-Y., Zhu, Y., Lazar, D., Fanti, G.: POPri: Private federated learning using preference-optimized synthetic data. In: Proceedings of the 42nd International Conference on Machine Learning (ICML) (2025) arXiv:2504.16438

[11] Chen, X., Shi, Y., Lan, Q., Qiu, Y., Wang, M., Gu, X., Yan, Y.: Fed-SE: Federated self-evolution for privacy-constrained multi-environment LLM agents. arXiv preprint (2025) arXiv:2512.08870

[12] Lin, Z., Hao, X., Fu, R., Cui, S., Chen, K., Li, C., Li, Z., Xiong, F.: A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle. arXiv preprint (2026) arXiv:2604.16548

[13] McMahan, H.B., Moore, E., Ramage, D., Hampson, S., Arcas, B.:

Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 54, pp. 1273–1282 (2017) arXiv:1602.05629

[14] Kairouz, P., Liu, Z., Steinke, T.: The distributed discrete Gaussian mechanism for federated learning with secure aggregation. In: Proceedings of the 38th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 139, pp. 5201–5212 (2021) arXiv:2102.06387

[15] Abadi, M., Chu, A., Goodfellow, I., McMahan, H.B., Mironov, I., Talwar, K., Zhang, L.: Deep learning with differential privacy. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 308–318 (2016) arXiv:1607.00133. https://doi.org/10.1145/2976749.2978318

[16] Bassily, R., Smith, A., Thakurta, A.: Private empirical risk minimization: Efficient algorithms and tight error bounds. In: IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS), pp. 464–473 (2014) arXiv:1405.7085. https://doi.org/10.1109/FOCS.2014.56

[17] Chen, W.-N., Choquette-Choo, C.A., Kairouz, P., Suresh, A.T.: The fundamental price of secure aggregation in differentially private federated learning. In: Proceedings of the 39th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 162, pp. 3056–3089 (2022) arXiv:2203.03761

[18] Bun, M., Steinke, T.: Concentrated differential privacy: Simplifications, extensions, and lower bounds. In: Theory of Cryptography Conference (TCC). Lecture Notes in Computer Science, vol. 9985, pp. 635–658 (2016) arXiv:1605.02065. https://doi.org/10.1007/978-3-662-53641-4_24 . Pure ε-DP implies $(\varepsilon^2/2)$-zCDP; used to compose the clip-selection cost

[19] Smith, A.: Privacy-preserving statistical estimation with optimal convergence rates. In: Proceedings of the 43rd Annual ACM Symposium on Theory of Computing (STOC), pp. 813–822 (2011). https://doi.org/10.1145/1993636.1993743 . Exponential-mechanism quantile selection

[20] Mironov, I.: Rényi differential privacy. In: IEEE 30th Computer Security Foundations Symposium (CSF), pp. 263–275 (2017) arXiv:1702.07476. https://doi.org/10.1109/CSF.2017.11

[21] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., Yu, D.: LongMemEval: Benchmarking chat assistants on long-term interactive memory. In: International Conference on Learning Representations (ICLR) (2025) arXiv:2410.10813

[22] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese

BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992 (2019) arXiv:1908.10084. https://doi.org/10.18653/v1/D19-1410 . Model: all-MiniLM-L6-v2